Photonic Computing Multidisciplinary Project

Yosef Ben Chedly - Report
Politecnico di Milano

MSc in High Performance Computing Engineering
a.y. 2025–2026

Abstract

We present a Scientific Machine Learning algorithm for solving one-dimensional boundary-value problems, demonstrated on the 1D Poisson equation, using an interferometer-mesh optical network simulated with the `neuroptica` library [1]. The unknown solution is approximated by a physically parameterized neural network composed of cascaded Optical Interference Units with holomorphic complex activations applied digitally between layers. The problem is translated in an energy minimization problem via the Deep Ritz Method. To align optimization with the photonic hardware, we implement recursive on-chip differentiation that analytically propagates spatial derivatives through the network without finite-difference approximations. Gradients with respect to optical phase-shifter parameters are obtained *in situ* through an adjoint-variable method: a physically injected adjoint field enables end-to-end training of the mesh. This framework provides a compact, physics-informed approach to differential equation solving on photonic platforms.

Keywords: Differential problems, Scientific Machine Learning, Optical Neural Networks, Deep Ritz method; on-chip training.

Contents

1 Introduction **4**

2 Scientific Machine Learning **5**
2.1 Physics-based models 5
2.1.1 Variational (weak) formulations 6
2.2 Data driven models: the case of Artificial Neural Networks . . 7
2.2.1 The Perceptron 7
2.2.2 Feed-Forward Neural Networks 8
2.2.3 Network Architecture 9
2.2.4 Approximation Error in FFNNs 9
2.3 Combined models 11
2.3.1 Early work on neural PDE solvers 11
2.3.2 Motivation for the Deep Ritz Method 12
2.3.3 Limitations 14

3 Deep Ritz method **15**
3.1 Variational problems 15
3.2 Ingredients of the Deep Ritz algorithm 15
3.3 Building trial functions 16
3.4 Deep Ritz formulation for elliptic problems 17
3.5 Quadrature by random sampling 17

4 Photonic Neural Networks **20**
4.1 Meshes of *MZIs* as building blocks 20
4.1.1 Clements topology 21
4.1.2 The role of topological symmetry 22
4.2 Mathematical description of a training iteration 23
4.2.1 Permittivities 24
4.2.2 The analog counterpart of the backpropagation algorithm 25
4.2.3 The adjoint variable method for on-chip gradient computation 27
4.2.4 Measuring the overlap between the forward and adjoint fields 31

5 Physics-informed Photonic Neural Networks **35**
5.1 The Poisson case 37
5.1.1 Photonic Deep Ritz Algorithm 40
5.2 Analytic computation of spatial derivatives 40
5.2.1 Computing the Neural Network's Jacobian 41

6 Benchmarks **43**
6.1 1D Poisson problems 43
6.1.1 Homogeneous Dirichlet boundary conditions 43
6.1.2 Neumann boundary condition at $x = 1$ (mixed BCs) . . 44
6.1.3 Notebook setup and analysis 46
6.1.4 1D benchmarks 47
6.1.5 Energy loss interpretation and strong-form residual . . 49

7 Conclusions and future work **54**
7.1 Future work 54
7.2 Conclusions 56

1 Introduction

The development of novel algorithms on optical architectures has shown the potential to provide innovative solutions to engineering-relevant problems ([2], [3], [4]).
The aim of this project is to investigate whether the combination of physics-based and data-driven mathematical models, together with its deployment on photonic architectures, can provide a truly new computational paradigm for solving differential equations.
The motivation of this work is both computational and resource-driven: complex simulations can require substantial resources on traditional HPC systems. While such applications represent the long-term vision, this work focuses on establishing a proof of concept by developing a photonic solver for simpler equations, demonstrating the potential and limitations of this approach before scaling to more complex problems.

2 Scientific Machine Learning

Scientific Machine Learning (SciML) is a multidisciplinary field that combines *physics-based models* and *data-driven approaches* for the solution of differential problems.
Physics-based models follow Galileo's scientific methodology: starting from the observation of a physical phenomenon, the underlying physics is understood, formulated in mathematical terms and then solved either analytically or numerically. Data-driven models, on the other hand, extract complex patterns through the statistical modeling of recorded data and exploit these patterns to make predictions. Data-driven methods have gained enormous popularity in recent years due to the availability of so-called *big data* and the widespread access to advanced high-performance computing platforms.
Scientific Machine Learning aims to bridge these two paradigms by injecting physical and mathematical knowledge into machine-learning algorithms. In this context, machine learning enhances the descriptive and predictive capabilities of physics-based models, motivating the investigation of SciML strategies for the solution of problems governed by *partial differential equations*.
Multiple definitions of Scientific Machine Learning coexist in the literature. Here, we adopt the definition most suited to our setting [5]:

> *"Scientific Machine Learning is an interdisciplinary field empowered by the synergy of physics-based computational models with machine-learning algorithms for scientific and engineering applications."*

2.1 Physics-based models

Physics-based models aim at describing physical phenomena by explicitly encoding the underlying laws of physics into mathematical form. Typically, these laws are expressed in terms of *differential equations*, whose solution represents the physical state of the system. Two main classes of problems can be distinguished.

- **Time-independent (steady or stationary) problems**, where the unknown solution depends only on space,

$$u = u(x), \qquad x \in \Omega,$$

  with $\Omega \subset \mathbb{R}^D$, $D = 1, 2, 3$, denoting the spatial domain.

- **Time-dependent problems**, where the solution evolves in time,

$$u = u(x, t), \qquad x \in \Omega,\ t \geq 0.$$

In a general setting, a physics-based model can be formulated as a parametric problem $\mathcal{P}$ of the form:

$$\mathcal{P}(u, d) = 0,$$

where u is the unknown solution and d denotes a set of input data or parameters (such as material properties, source terms or boundary and initial conditions). For each admissible choice $d \in \mathcal{D}$, the problem admits a solution u. A fundamental requirement of well-posedness of $\mathcal{P}$ is the *continuous dependence of the solution on the data.*

More precisely, changing the data d does not alter the structure of the physical problem, but only affects its solution. This reflects the causal nature of physics-based models: variations in the inputs produce variations in the outputs.

As a simple example, consider the following one-dimensional boundary value problem:

$$\begin{cases} -u''(x) = f(x), & x \in (0, 1), \\ u(0) = u(1) = 0, \end{cases}$$

where the data are given by the source term f and the spatial domain $\Omega = (0, 1)$, with the related boundary conditions. In this case, the problem operator is fixed, while different choices of f lead to different solutions u.

From a physical perspective, this model describes the equilibrium configuration of an elastic string fixed at both ends. The function $u(x)$ represents the vertical displacement of a point of the string, while the forcing term $f(x)$ corresponds to an external load acting on the system.

2.1.1 Variational (weak) formulations

The weak formulation is preferred over the strong form in classical numerical analysis because it relaxes the differentiability requirements. To build intuition, consider the one-dimensional Poisson problem on $\Omega = (0, 1)$ with homogeneous Dirichlet boundary conditions ($u(0) = u(1) = 0$) and a concentrated forcing term:

$$u''(x) = \delta(x - x_0)$$

where δ denotes the Dirac distribution and $x_0 \in (0, 1)$. The corresponding solution is continuous but not twice differentiable, exhibiting a corner point at x_0. Physically, this problem is analogous to an elastic string fixed at both ends and pushed at a single point by a finger: the displacement of the string remains continuous, while the slope changes suddenly at the point where the force is applied. In this setting, the strong form fails because the second derivative does not exist in the classical sense (and not even the first one), whereas the weak formulation remains well-defined.

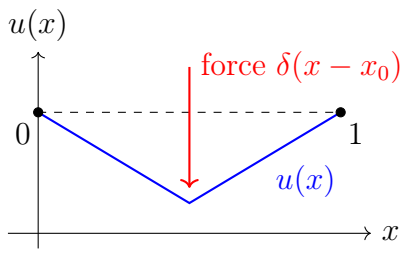

Figure 1: Graphical representation of the 1D Poisson problem with a concentrated forcing term. The solution $u(x)$ is continuous but not twice differentiable in $(0, 1)$, reflecting the discontinuity in the derivative at the point of the Dirac forcing.

2.2 Data driven models: the case of Artificial Neural Networks

A Machine Learning (*ML*) model is a mathematical function, and as such, it maps an input $\mathbf{x} \in \mathcal{X} \subseteq \mathbb{R}^n$ to an output $\mathbf{y} \in \mathcal{Y} \subseteq \mathbb{R}^m$ and depends on parameters $\mathbf{w} \in \mathcal{M} \subseteq \mathbb{R}^p$. The mapping mechanism is chosen tuning the model's parameters.

Definition (ML model)

$$\mathbf{f}(\mathbf{x}|\mathbf{w}) : \mathcal{X} \times \mathcal{M} \subseteq \mathbb{R}^{n+p} \longrightarrow \mathcal{Y} \subseteq \mathbb{R}^m,$$

where $\mathcal{X}$ is the *input space*, $\mathcal{Y}$ the *target space*, $\mathcal{M}$ the *parameter space*. $\mathcal{Y}^{\mathcal{X}}$ is the space where all functions $\mathbf{f} : \mathcal{X} \longrightarrow \mathcal{Y}$ live (deterministic mapping).

Since this space is infinite-dimensional, the search for the model is more conveniently restricted to a subset $\mathcal{H} \subset \mathcal{Y}^{\mathcal{X}}$, called the *hypothesis space*. Each choice of $\mathcal{H}$ corresponds to a specific family of ML models: we're choosing Artificial Neural Networks (*ANNs*).

2.2.1 The Perceptron

Artificial neurons mimic the functioning of biological ones. A biological neuron receives electrical impulses from other connected neurons through structures called dendrites, processes the information in its cell body and, if the result of the computation exceeds a certain threshold, sends a new impulse to subsequent neurons through the axon and synapses. Otherwise, the signal is suppressed and subsequent neurons do not receive any output

from this neuron.
Perceptrons are mathematical models that aim to reproduce the aforementioned correspondence. A perceptron is the fundamental non-linear computational unit of a *Neural Network*: it is the artificial neuron.
Given input features x_j, weights w_j, and a bias b, the perceptron computes:

$$h = \sum_j w_j x_j + b,$$

and produces the output

$$y = \sigma(h),$$

where σ is the *activation function*, a continuous approximation of the Heaviside step function

$$H(h) = \begin{cases} 1, & h > 0, \\ 0, & h \leq 0. \end{cases}$$

The weights w_j represent the connection strengths between neurons, while b determines the activation threshold (with opposite sign). The perceptron, conceived as a mathematical model, reads:

$$y = f(\mathbf{x}|\mathbf{w}) = \sigma\left(\sum_j w_j x_j + b\right), \tag{1}$$

where

$$\mathbf{w} = [w_1, w_2, \ldots, w_n, b]$$

is the set of model's parameters.

2.2.2 Feed-Forward Neural Networks

A *Feed-Forward Neural Network* (*FFNN*) consists of multiple layers of perceptrons where the output of one layer becomes the input to the next. Intermediate layers are called *hidden layers*. The number of hidden layers quantifies the depth of a neural network. Let L denote the number of layers (including the input and output layers) and N_ℓ the number of neurons at layer $\ell \in \{0, 1, \ldots, L\}$. We define:

$$W^{(\ell)} \in \mathbb{R}^{N_\ell \times N_{\ell-1}}, \quad \mathbf{b}^{(\ell)} \in \mathbb{R}^{N_\ell},$$

as respectively the weight matrix and bias vector of layer ℓ.
Given the input $\mathbf{a}^{(0)} = \hat{\mathbf{x}}$, the forward propagation is recursively defined by

$$\begin{aligned} \mathbf{h}^{(\ell)} &= W^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \\ \mathbf{a}^{(\ell)} &= \sigma(\mathbf{h}^{(\ell)}), \end{aligned}$$

for $\ell = 1, \ldots, L$, with final output:

$$\mathbf{y} = \mathbf{f}(\hat{\mathbf{x}}|\mathbf{w}) = \mathbf{a}^{(L)}.$$

Hence, the *FFNN realization function* can be expressed as a nested composition:

$$\mathbf{f} = \sigma \circ T^{(L)} \circ \sigma \circ T^{(L-1)} \circ \cdots \circ \sigma \circ T^{(1)},$$

where $T^{(\ell)}(\mathbf{x}) = W^{(\ell)}\mathbf{x} + \mathbf{b}^{(\ell)}$.

2.2.3 Network Architecture

The architecture of a neural network is characterized by:

- L: number of layers (depth),
- $\{N_\ell\}$: number of neurons per layer (width),
- σ: activation function (non-linearity).

These quantities are called *hyperparameters*. These are different from the *parameters* of the network, i.e., weights and biases, that we want to optimize by minimizing a loss function. Hyperparameters control the model's parameters, determining how these parameters are learned during training.

2.2.4 Approximation Error in FFNNs

The *approximation error* measures the expressivity of the ML model, i.e. how well a given input-output function can be approximated by functions in the (hypothesis) space spanned by the chosen neural network architecture. For Feed Forward Neural Networks (FFNNs), we distinguish between *shallow* and *deep* architectures. Shallow FFNNs ($L = 2$, one hidden layer) are *universal approximators* according to the Universal Approximation Theorem. This means that, under suitable regularity assumptions on the activations, any continuous function on a compact set can be approximated by a suitable shallow FFNN up to arbitrary precision.
In practise, some activation functions, like ReLU or sigmoid (strong non-linearities), are expressive enough to let a neural network approximate a very wide variety of functions, while simple polynomials (weak non-linearities) are not rich enough for this task (it can be proved). A shallow network with just one hidden layer can, in principle, represent any function if it has enough neurons, but the number of neurons needed increases rapidly when the function is more complex, when higher accuracy is required or

when the input has many dimensions. Deep networks, with multiple hidden layers, can achieve similar or better approximations with fewer total neurons, because the layers can build hierarchical, compositional representations of the input. In particular, deep FFNNs are typically more efficient: for a given approximation tolerance, the total number of non–null parameters p needed is lower than in shallow networks. Moreover, with the same number of parameters p, deep networks can represent more complex functions than shallow networks. Even so, no network can completely avoid the intrinsic difficulty of high-dimensional problems: there is always a minimal number of neurons and parameters required. Practically, for simple, low-dimensional problems, shallow networks may work well; for more complex or structured functions, deep networks are more efficient and can reach the desired accuracy with fewer resources.

2.3 Combined models

2.3.1 Early work on neural PDE solvers

The first documented attempt to solve partial differential equations (PDEs) using neural networks dates back to 1994 [6]. The authors proposed a feed-forward neural network with sigmoid activation functions to approximate the solution of a boundary-value problem of the following form:

$$\begin{cases} Au = f, & \text{in } \Omega, \\ Du = g, & \text{on } \partial\Omega, \end{cases} \tag{2}$$

where:

- Ω is an open bounded domain with boundary $\partial\Omega$,
- f is a given source term,
- A and D are suitable differential and boundary operators representing the PDE and boundary conditions.

They stated explicitly[1]:

> "*A so-called strong solution of (2) is an element of some underlying space* U *which satisfies both the PDE and the boundary conditions. [...] However, if there is a 'universal' approximator in this underlying space, that is, a function that has enough parameters and can be arbitrarily close to any element of the space by a judicious choice of the parameters, then this approximator can be a good choice for the approximate solution sought for. The closeness of two functions is described by the concept of denseness in* U."

Following this idea, the PDE-solving task was formulated as the minimization of an objective function:

$$\mathcal{F}(u_{\text{NN}}) = \|Au_{\text{NN}} - f\|_{\Omega}^2 + \|Du_{\text{NN}} - g\|_{\partial\Omega}^2, \tag{3}$$

where u_{NN} is the neural network approximation of the solution and $\|\cdot\|_{\Omega}$ and $\|\cdot\|_{\partial\Omega}$ are suitable norms measuring the residuals in the domain and on the boundary, respectively. The minimization of the residual functional written in eq. (3) is a nonlinear problem over the neural network parameters θ:

$$\theta^* = \arg\min_{\theta} \mathcal{F}(u_{\text{NN}}). \tag{4}$$

[1] Original symbols and references have been adapted for clarity. "[...]" indicates omitted text.

2.3.2 Motivation for the Deep Ritz Method

Building on the previous formulation of a PDE problem as a minimization problem over neural network parameters θ, the *Deep Ritz Method* [7] applies the same idea using the *variational (Ritz) formulation* of PDEs.
The weak formulation is generally preferred over the strong form because it relaxes the regularity requirements on the solution and provides empirically a more stable and flexible mathematical framework.
In the strong form, the solution is required to satisfy the differential equation pointwise and requires high regularity of the solution (e.g. C^2 continuity), as occurs with elliptic problems. In contrast, the weak formulation uses integration by parts to lower the requirement over the regularity of the unknown. Moreover, integration by parts naturally incorporates Neumann (natural) boundary conditions into the formulation, avoiding the need to enforce them explicitly as in the strong form.
By rewriting the problem in an integral (weighted-average) form, the weak formulation enforces the governing equation in an averaged sense over the domain rather than pointwise. This significantly improves robustness with respect to irregular geometries, nonsmooth coefficients and discontinuous boundary conditions. Empirical evidence in the context of Physics-Informed Neural Networks suggests that variational formulations based on the weak form typically require fewer collocation or quadrature points than strong-form residual minimization methods to achieve comparable accuracy, though a rigorous theoretical quantification of this advantage remains an open question. Nevertheless, we can intuitively think of the weak formulation as an integration process that inherently acts as a filter, reducing noise and providing more stable, accurate approximations with equivalent computational resources.

In particular, when the bilinear form associated with the differential operator is symmetric, the weak formulation is equivalent to the Euler–Lagrange condition of an energy functional: the differential problem is encoded in an energy minimization problem (Ritz method). However, many relevant problems involve non-symmetric operators (e.g. advection problems), for which no underlying energy functional exists. In such cases, classical Ritz-based approaches are no longer applicable.
However, there are methods, like the *Deep Double Ritz Method* (D^2RM), that extend the variational framework to non-symmetric problems, allowing to retain the advantages of the weak, integral formulation [8].

Example:
Let Ω be an open bounded domain and consider the sufficiently regular

functions $f : \Omega \to \mathbb{R}$, $h : \partial\Omega \to \mathbb{R}$ and a real coefficient $\gamma > 0$: we look for the solution $u : \Omega \to \mathbb{R}$ of the elliptic equation with Neumann boundary conditions that follows [2]

$$\begin{cases} -\Delta u + \gamma u = f & \text{in } \Omega \\ \nabla u \cdot n = g & \text{on } \partial\Omega. \end{cases} \tag{5}$$

After introducing the properly defined space V, the weak form reads [3]:

$$\text{Find } u \in V : \quad \int_\Omega (\nabla u \cdot \nabla v + \gamma u v)\, dx = \int_\Omega f v\, dx + \int_{\partial\Omega} g v\, d\gamma \quad \forall v \in V$$

or, equivalently,

$$\text{Find } u \in V : \quad a(u, v) = F(v) \quad \forall v \in V.$$

When the bilinear form $a(u, v)$ associated with the differential operator is symmetric[4] like the one of the problem (5), the weak form of the PDE can be interpreted as the Euler-Lagrange equation of the following energy functional:

$$\mathcal{E}(v) = \frac{1}{2} \int_\Omega |\nabla v|^2 + \frac{\gamma}{2} \int_\Omega v^2 - \int_\Omega f v - \int_{\partial\Omega} g v \quad \forall v \in V,$$

so that

$$u = \arg\min_{v \in V} \mathcal{E}(v).$$

With this example, we aimed to illustrate the process of the weak formulation derivation, including reduced regularity requirements, visually show an instance of symmetric bilinear form, the concept that the PDE has to be satisfied in an average sense over the physical domain, the natural incorporation of Neumann boundary conditions into the energy formulation of the problem. We emphasize that, while symmetry of the bilinear form is required in this setting, other methods have been developed to extend the approach to non-symmetric bilinear forms, such as those arising from convection terms in fluid dynamics problems.

In summary, we select the Deep Ritz Method for our photonic solver because this approach is potentially advantageous in high-dimensional problems, where pointwise enforcement of the strong form leads to severe instability and the curse of dimensionality.

[2] For the well-posedness analysis of this problem, we refer the reader to standard textbooks on numerical analysis, such as [9].

[3] We notice the reduced regularity requirement on u: in the new formulation of our problem, we are asked to compute only its first derivative, not the second one and the natural encoding of boundary conditions.

[4] $a(u, v) = a(v, u)$.

2.3.3 Limitations

The limitations of the Deep Ritz Method are well known and are discussed in detail in [5]. For an in-depth analysis tailored to the present work, we refer the reader to section 7.

3 Deep Ritz method

We are interested in solving variational problems that arise from partial differential equations (PDEs). To do so, we resort to the **Deep Ritz method** (introduced in [7]), that is a *deep learning-based* numerical algorithm. Its name is due to the fact that neural networks are used to represent functions in the context of the *Ritz method*.

Deep learning techniques introduce a new way to approximate functions, establishing an alternative paradigm to the well-known *basis expansion methods*. In contrast to *additive construction*, where a function is expressed as a linear combination of predefined basis elements, the *compositional construction* technique builds complex functions hierarchically through a nested composition of simpler functions. Although additive methods are effective for the approximation of low-dimensional smooth functions, compositional techniques seem to be powerful in high dimensions [7]. Many works focus on the solution of PDEs using artificial neural networks, including [8], which suggests that neural networks are particularly suitable in contexts where functions need to be constructed, including the solution of problems governed by PDEs [7].

3.1 Variational problems

The variational problems we are interested in can be formulated as follows:

$$\textit{Find } u^* \in \mathcal{H} \mid u^* = \min_{u \in \mathcal{H}} I(u), \tag{6}$$

where $\mathcal{H}$ is the set of admissible functions and $I(u)$ an integral that arises from the *weak formulation* of the original differential problem, defined inside the physical domain Ω.

3.2 Ingredients of the Deep Ritz algorithm

The *Deep Ritz procedure* needs the following ingredients [7]:

- A *neural network* to approximate u;
- A *quadrature rule* to approximate the integral $I(u)$;
- An *optimization algorithm* to solve the optimization problem.

3.3 Building trial functions

In this paragraph we present the strategy used in [7] to build *trial functions*, i.e. functions that live in $V_{\text{NN}} = \left\{ u(x) = u_{\text{NN}}(x|\theta) \,\middle|\, \theta \text{ are NN's parameters} \right\}$, the functional space spanned by the neural network (NN). The core component of the *Deep Ritz Method* is an artificial neural network, implementing a nonlinear transformation:

$$\mathbf{x} \in \mathbb{R}^d \to \mathbf{X}_L(x|\theta) \in \mathbb{R}^{n_{out}}.$$

In the original formulation, each layer of the network is constructed by stacking several *blocks*. In [7], the authors designed a deep neural network composed of two blocks (a tunable hyperparameter), each consisting of two linear layers with their corresponding activation functions and a residual (skip) connection in parallel, similar to the ResNet architecture [10]. Therefore, each block contains two linear transformations, two activation functions and a skip connection. Here, θ denotes the set of network parameters, i.e. the weights and biases associated with the connections between neurons, while n_{out} represents the dimension of the output layer, including the hidden ones within the network.

Each block of the network maps an n_{out} dimensional vector to another n_{out} dimensional vector. The choice of n_{out} is a hyperparameter controlling the network width and its expressive power[5]. Although the solution $u(x)$ is scalar-valued, the network uses intermediate vectors to efficiently capture complex structures of the function.
In the present setting, skip connections are neglected for simplicity. We denote with $\mathbf{p}$ and $\mathbf{q}$ respectively the input and output vectors of a block (both n_{out} dimensional). The i-th block can be expressed as:

$$\mathbf{q} = \phi_i(\mathbf{p}) = \underbrace{\phi\big[W_{i2}\ \phi\big(W_{i1}\mathbf{p} + \mathbf{b}_{i1}\big) + \mathbf{b}_{i2}\big]}_{\text{two linear layers + two non-linear activations}}, \tag{7}$$

where $W_{i1}, W_{i2} \in \mathbb{R}^{m\times m}$ and $\mathbf{b}_{i1}, \mathbf{b}_{i2} \in \mathbb{R}^m$ are the trainable parameters of the block and $\phi(\cdot)$ is a element-wise scalar activation function. The smoothness of the activation function plays a key role in the accuracy of the algorithm. The full L-layer network is then [6]:

$$f_{NN}(\theta) = f_L \circ f_{L-1} \circ \cdots \circ f_1. \tag{8}$$

[5]The *expressive power* of a neural network refers to its ability to approximate complex functions with a given architecture size, that is, how richly it can represent nonlinear mappings between inputs and outputs.

[6]Also L (depth) is an hyperparameter and affects the expressiveness of the network.

Note that the input $x \in \mathbb{R}^D$ of the first block may not match n_{out}, the dimension of the hidden vectors. To handle this, we need an encoding rule. Finally, the function $u(x|\theta)$ is obtained as through a linear readout layer:

$$u_{NN}(x|\theta) = \mathbf{a} \cdot \mathbf{X}_L(x|\theta) + b, \tag{9}$$

where $\mathbf{a} \in \mathbb{R}^{n_{out}}$ and $b \in \mathbb{R}$ either can be part of the full parameter set θ or be fixed a priori.

3.4 Deep Ritz formulation for elliptic problems

For elliptic problems, the variational formulation leads to an *energy minimization* problem. Let $\Omega \subset \mathbb{R}^d$ and let u belong to a suitable space V. We seek

$$u^\star \in \arg\min_{u \in V} I[u], \qquad I[u] = \int_\Omega g\big(x, u(x), \nabla u(x)\big)\, dx \; + \; \int_{\partial\Omega} b\big(x, u(\mathbf{x})\big)\, d\gamma. \tag{10}$$

In the Deep Ritz Method we approximate the unknown field with a neural network, so that the energy becomes a function of the parameters:

$$\min_\theta L(\theta), \qquad L(\theta) \equiv I[u_{NN}(\theta)]\,. \tag{11}$$

In particular, the integrand g depends on u_{NN} and its first derivative. Consistently with the graph of Figure 2, at each sampled point we must evaluate both u_{NN} and ∇u_{NN}. Indeed, both are needed to compute the contribution of the single sampled point to the loss.

3.5 Quadrature by random sampling

To compute (11) we approximate the integrals by random sampling. We draw N_{int} points $\{x_i^{\text{int}}\}_{i=1}^{N_{\text{int}}}$ in Ω and N_{bc} points $\{x_i^{\text{bc}}\}_{i=1}^{N_{\text{bc}}}$ on $\partial\Omega$. Using uniform weights (Monte Carlo quadrature), we obtain the empirical loss:

$$\begin{aligned} L(\mathbf{w}) \approx \frac{1}{N_{\text{int}}} \sum_{i=1}^{N_{\text{int}}} g\Big(x_i^{\text{int}},\, u_{NN}(x_i^{\text{int}}|\theta),\, \nabla u_{NN}(x_i^{\text{int}}|\theta)\Big) + \\ + \frac{1}{N_{\text{bc}}} \sum_{i=1}^{N_{\text{bc}}} b\Big(x_i^{\text{bc}},\, u_{NN}(x_i^{\text{bc}}|\theta)\Big). \end{aligned} \tag{12}$$

This has the standard machine-learning form $L(\theta) \approx \frac{1}{N}\sum_i L_i(\theta)$, so it can be minimized by with mini-batch gradient descent. At iteration k, let B_{int}^k

and B^k_{bc} be mini-batches of interior and boundary points, respectively. The update reads

$$\begin{aligned}\theta^{k+1} = \theta^k - \eta\,\nabla_\theta \Bigg[&\frac{1}{|B^k_{\text{int}}|} \sum_{x \in B^k_{\text{int}}} g\Big(x,\, u_{NN}(x|\theta^k),\, \nabla u_{NN}(x|\theta^k)\Big) + \\ &+ \frac{1}{|B^k_{\text{bc}}|} \sum_{x \in B^k_{\text{bc}}} b\Big(x,\, u_{NN}(x|\theta^k)\Big)\Bigg].\end{aligned} \tag{13}$$

Randomly resampling points at each iteration reduces the risk of overfitting a fixed quadrature grid and provides an unbiased estimate of the gradient of the energy functional.

Example:
Consider the Poisson problem

$$-\Delta u = f \quad \text{in } \Omega, \qquad u = \alpha \quad \text{on } \partial\Omega. \tag{14}$$

The Deep Ritz energy with penalty term is

$$I[u] = \int_\Omega \left(\frac{1}{2} |\nabla u|^2 - f\, u \right) dx + \frac{\beta}{2} \int_{\partial\Omega} (u - \alpha)^2 \, d\gamma, \tag{15}$$

where $\beta > 0$ is a penalty parameter. We set for simplicity $\alpha = 0$ (homogeneous Dirichlet problem). Replacing u by u_{NN} and approximating integrals by random sampling yields the loss:

$$L(\theta) \approx \frac{1}{N_{\text{int}}} \sum_{i=1}^{N_{\text{int}}} \left[\frac{1}{2} \left| \nabla u_{NN}(x_i^{\text{int}}|\theta) \right|^2 - f(x_i^{\text{int}})\, u_{NN}(x_i^{\text{int}}|\theta) \right] \tag{16}$$

$$+ \frac{\beta}{2N_{\text{bc}}} \sum_{i=1}^{N_{\text{bc}}} u_{NN}(x_i^{\text{bc}}|\theta)^2. \tag{17}$$

The described workflow is illustrated in Fig. 2.

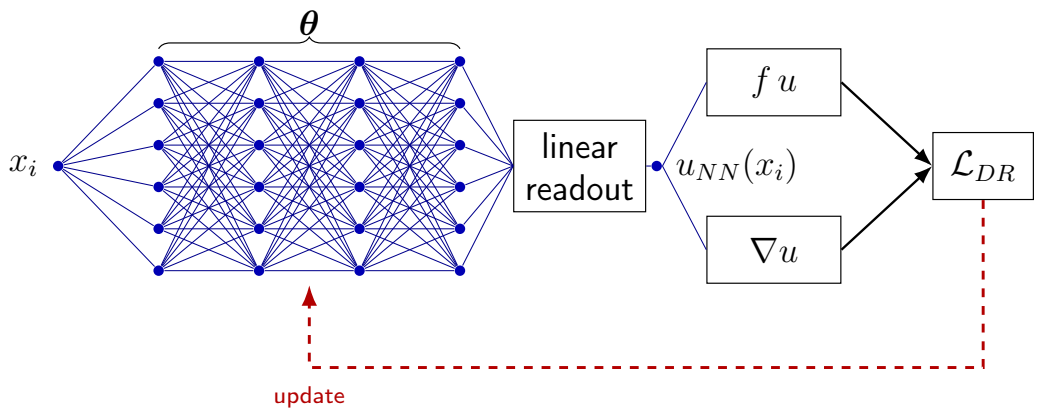

Figure 2: Workflow of the Deep Ritz Method. The loss evaluation at a sampled point x_i is illustrated: the network maps the input to a hidden representation, a linear readout produces the scalar output $u_{NN}(x_i)$, $\nabla u_{NN}(x_i)$ is provided. These quantities are then combined to form the loss contributions (here shown schematically via $f\,u$ and ∇u) and aggregated into $\mathcal{L}_{DR}$. The red dashed arrow represents parameters' update.

4 Photonic Neural Networks

Integrated optics offers interesting novel hardware to implement machine learning algorithms. An in-situ training approach for our photonic neural network is employed. The method presented in [11] takes advantage of the *adjoint variable method* to derive the photonic counterpart of the backpropagation algorithm.

We have seen that a *FFNN* transforms an input vector into an output vector through alternating linear transformations and element-wise nonlinear activation functions. A cost function is defined over the outputs of the ANN and is minimized by tuning the matrix elements involved in the linear operations. This is done through gradient-based optimization, where gradients are computed using the backpropagation algorithm.
One of the most relevant attempts to build a photonic neural network was presented in [12]. Linear operations are performed through *Optical Interference Units* (*OIUs*), which are meshes of *Mach-Zender Interferometers* (*MZIs*), that can implement any $N \times N$ unitary trasformation by tuning the integrated phase shifters. Non-linear activations can be performed using either electronic circuits or digital computers; this implies that the optical state must be measured before activation and converted to an electrical signal for nonlinear processing, then re-prepared in a transformed optical state to be injected into the next *OIU* of the *photonic FFNN*.

4.1 Meshes of *MZIs* as building blocks

OIUs are made of a number of single-mode waveguide input ports and, in general, of another number of single-mode output ports, coupled together through an ideally linear and lossless device. The single *OIU* implements the following linear operation:

$$\hat{W}\,\mathbf{X}_{in} = \mathbf{Y}_{out}, \tag{18}$$

where $\mathbf{X}_{in}$ and $\mathbf{Y}_{out}$ are the modal fields' amplitudes at the input and output ports, respectively and the matrix $\hat{W}$ is the linear operator that can be tuned by acting on the integrated phase shifters of the device. For reasons that will become clearer later, we need to provide the most general form of this linear operation, since we must work with the same device in both the backward and forward passes:

$$\begin{pmatrix} \mathbf{X}_{out} \\ \mathbf{Y}_{out} \end{pmatrix} = \begin{pmatrix} \mathbf{0} & \mathbf{W}^T \\ \mathbf{W} & \mathbf{0} \end{pmatrix} \begin{pmatrix} \mathbf{X}_{in} \\ \mathbf{Y}_{in} \end{pmatrix} \tag{19}$$

The diagonal blocks are zero because we assume forward-only propagation when optical signals are sent through the input ports and backward-only propagation when signals are sent through the output ports. Another assumption we make is that of a *reciprocal device*, meaning that the modal amplitudes at the output ports experience the transpose of the transfer matrix seen by the input modal amplitudes. This is an assumption for the mathematical discussion of the device's working principle, as well as a real practical design specification of the physical platform (see Fig. 3).

4.1.1 Clements topology

An $N \times N$ universal linear unitary operator can be reconstructed using the so-called *Clements decomposition*. This mathematical scheme is physically implemented with a *Clements topology*, a rectangular mesh which requires $\frac{N(N-1)}{2}$ Mach–Zehnder interferometers (*MZIs*). The latter is the minimal number that allows us to realize any unitary matrix (under the hypothesis of lossless devices), given that each phase shifter has 2 tunable parameters and a phase shifter layer is provided at the beginning or at the end of the mesh (one phase shifter per input or output port). A *Clements mesh* represents a building block for our *Photonic Neural Network*; therefore we will call it an *Optical Interference Unit* (*OIU*). The `neuroptica` library provides exactly this architecture.

Each MZI is composed exclusively of passive[7] and reciprocal [8] elements (directional couplers and phase shifters). As a consequence, the full photonic circuit is reciprocal. A composite device that is made only of reciprocal devices is reciprocal itself. Reciprocity implies that backward propagation through the same device (injection of an optical signal from the output ports) implements the transpose of the forward operator:

$$\hat{W}_{\text{backward}} = \hat{W}^T, \tag{20}$$

[7]An optical element is *passive* if it does not add net energy to the optical field. All linear, lossless passive devices conserve optical power.

[8]An optical element is *reciprocal* if its response is invariant when the direction of light propagation is reversed (time-reversal symmetry). Most linear passive components (waveguides, directional couplers, phase shifters) are reciprocal unless designed to break time-reversal symmetry.

with $\hat{W}$ being the transfer matrix of the device in the forward propagation setting. Importantly, this property is guaranteed by reciprocity alone and does not depend on any geometric or topological symmetry of the layout.

4.1.2 The role of topological symmetry

We said that reciprocity guarantees the transpose operation. This is true at the cost of some permutation of the optical signals at the output ports.
Introducing a topologically symmetric layout (e.g. mirrored mesh) does not alter reciprocity, but avoids channel reordering at the output ports. In numerical frameworks such as `neuroptica`, channel permutations are handled entirely in software. The transpose W^T is computed numerically and applied digitally, making hardware symmetry unnecessary. However, when training is performed *in situ*, as it is in optical backpropagation experiments, one has to consider that channel permutations correspond to real photonic operations which can introduce: additional optical loss, uncontrolled phase offsets, increased crosstalk and degraded signal-to-noise ratio. These effects would directly affect learning stability and system's fidelity.
To avoid optical channel permutations, a topologically symmetric mesh can be adopted. This choice requires an overparameterization, adding $O(N)$ MZIs beyond the minimal $\frac{N(N-1)}{2}$ required for universality [9], as shown in Fig. 3. This overparameterization is not required mathematically, but is physically motivated by reduced optical scheduling complexity. Thus, the design trades a small increase in hardware complexity for improved signal quality and robust physical backpropagation. In summary:

- `neuroptica` (software): minimizes hardware complexity, accepts software scheduling and does not exploit physical backward propagation.
- hardware: suggests overparameterization to eliminate channel permutations and enable stable in-situ optical learning.

These approaches are not contradictory but optimized for fundamentally different operating regimes, as done in [11].

[9] The `ClementsLayer` constructor provided by `neuroptica` includes an optional `PhaseShifterLayer(N)` at the mesh input. This layer provides the N global phase degrees of freedom required, in conjunction with the (θ, ϕ) pairs in each *MZI*, to implement an arbitrary $N \times N$ unitary matrix. When `include_phase_shifter_layer=True` (default), the total parameter count is $2 \times \frac{N(N-1)}{2} + N$, achieving full universality while maintaining only two phase shifters per physical MZI.

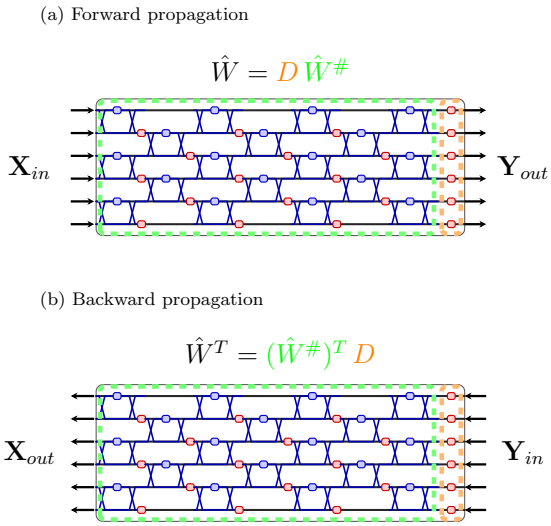

Figure 3: A rectangular Clements mesh $\hat{W}^{\#}$ ($N = 6$) followed by a diagonal phase-shifter layer D. (a) Forward propagation implements $\hat{W}$. (b) By reciprocity, backward propagation yields W^T. Since D is diagonal, $D^T = D$, so the final single-mode phase layer does not break topological symmetry and requires no waveguide permutations, while allowing universality.

4.2 Mathematical description of a training iteration

A training iteration is made of forward and backward passes. In the forward propagation step, we send an input $\mathbf{X}_0$ to the system and perform a linear operation represented by the matrix $\hat{W}_1$, followed by the application of a element-wise non linear activation $\boldsymbol{\phi}_1$ on the result, providing the input for the next layer. This process is repeated for each layer ℓ until the last ouput layer L, as shown in Fig. 4. Therefore:

$$\mathbf{X}_\ell = \boldsymbol{\phi}_\ell(\hat{W}_\ell\,\mathbf{X}_{\ell-\mathbf{1}}) = \boldsymbol{\phi}_\ell(\mathbf{Y}_\ell). \tag{21}$$

Finally the cost function depends on the outputs from the last layer of the optical network:

$$\mathcal{L} = \mathcal{L}(\mathbf{X}_L). \tag{22}$$

To train the network, we have to minimize the cost function with respect to the linear operators $\hat{W}_\ell$ (see the next subsection for details).
In conventional photonic training schemes, the gradients of the cost function are computed *ex situ*, relying on a digital model of the optical system (ref. [12]). In this framework, the optical circuit performs only the physical forward pass, while the optimization of the parameters is carried out digitally.

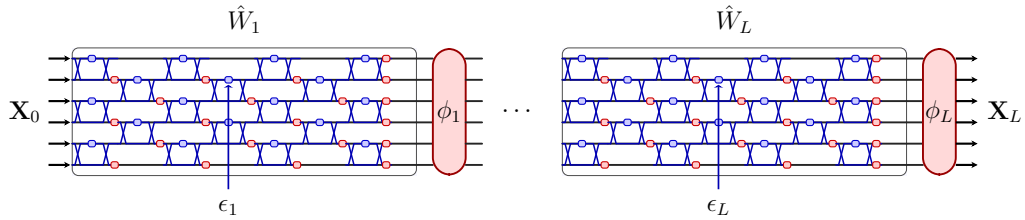

Figure 4: Schematic representation of the photonic neural network architecture. Each *OIU* is followed by a non-linear activation layer. The hyperparameter depth is L and phase shifters are the network's parameters.

Specifically, the forward propagation is executed in analog hardware, the output is physically measured and converted into a digital signal for the evaluation of the error. The optical circuit has no access to the target values, which are exclusively handled by the external computer. The optimization step is therefore performed in the digital domain, the updated weights are converted back into physical control signals on the phase shifters and the new parameters are uploaded onto the chip for the next iteration. This continuous analog–digital–analog conversion constitutes a major computational bottleneck and, ultimately, does not provide a substantial advantage over fully digital training strategies, since the continuous conversion between the analog and digital domains nullifies the advantages of high-speed optical computations.
The same reference also proposes an *in situ* method for gradient estimation (ref. [12]). However, this approach scales linearly with the number of tunable parameters, since it is based on sequentially perturbing one phase shifter at a time and measuring the corresponding variation of the output.
By contrast, true *in situ* training is achieved by the approach presented in ref. [11], where both forward and backward propagation are physically implemented on the photonic hardware. In this work, the weight updates are computed directly on chip, enabling a massively parallel evaluation of the gradients, therefore scaling significantly better with respect to the number of parameters when compared to the brute force gradient computation.

4.2.1 Permittivities

In the following we denote ε_ℓ the relative dielectric constant (or permittivity) of a single, arbitrary chosen phase shifter in the ℓ-th layer, as the same following mathematical discussion holds for the other phase shifters present in that layer.
We said that, in order to train the network, we have to minimize the cost

function with respect to the linear operators $\hat{W}_\ell$. Since the latter operator explicitly depends on all the related permettivities ϵ_ℓ, this means that the gradient of the loss function has to be computed with respect to the permettivities of the on-chip phase shifters.

Permittivity and phase shifting

The permittivity of a phase shifter refers to the (relative) dielectric constant ε_r of the material. For non-magnetic, lossless optical materials, the refractive index n is directly related to the permittivity by $n \approx \sqrt{\varepsilon_r}$. A phase shifter applies a controllable optical phase ϕ to the propagating signal. Any electrically or thermally induced variation of ε_r produces a change in the effective refractive index and thus a variation of the applied phase.

4.2.2 The analog counterpart of the backpropagation algorithm

We now revisit the procedure described in ref.[11] to derive the on-chip gradient measurement mechanism, which is the key enabler for in-situ training. We follow a *supervised learning* approach. We take the following Mean Squared cost function:

$$\mathcal{L}_{\text{MS}} = \frac{1}{2}\|\mathbf{X}_L - \mathbf{T}\|^2 = \frac{1}{2}(\mathbf{X}_L - \mathbf{T})^\dagger(\mathbf{X}_L - \mathbf{T}) \tag{23}$$

where $\mathbf{T}$ is the complex target vector given input $\mathbf{X}_0$.
Following the backpropagation algorithm [13], starting from the last layer of the network, the derivative of the cost function with respect to the permittivity ε_L is given by:

$$\begin{aligned}
\frac{\partial \mathcal{L}_{\text{MS}}}{\partial \varepsilon_L} &= \frac{\partial}{\partial \varepsilon_L}\left[\frac{1}{2}(\mathbf{X}_L - \mathbf{T})^\dagger(\mathbf{X}_L - \mathbf{T})\right] = \\
&= \frac{1}{2}\left[\frac{\partial(\mathbf{X}_L - \mathbf{T})^\dagger}{\partial \varepsilon_L}(\mathbf{X}_L - \mathbf{T}) + (\mathbf{X}_L - \mathbf{T})^\dagger\frac{\partial(\mathbf{X}_L - \mathbf{T})}{\partial \varepsilon_L}\right] = \\
&= \text{Re}\left\{(\mathbf{X}_L - \mathbf{T})^\dagger\frac{\partial \mathbf{X}_L}{\partial \varepsilon_L}\right\}.
\end{aligned}$$

Since $\mathbf{X}_L = \boldsymbol{\phi}_L(\hat{W}_L \mathbf{X}_{L-1})$ and $\mathbf{X}_{L-1}$ does not depend on ε_L, we apply the chain rule:

$$\begin{aligned}\frac{\partial \mathbf{X}_L}{\partial \varepsilon_L} &= \frac{\partial \boldsymbol{\phi}_L(\hat{W}_L \mathbf{X}_{L-1})}{\partial \varepsilon_L} = \\ &= \boldsymbol{\phi}'_L(\hat{W}_L \mathbf{X}_{L-1}) \frac{\partial (\hat{W}_L \mathbf{X}_{L-1})}{\partial \varepsilon_L} = \\ &= \boldsymbol{\phi}'_L(\hat{W}_L \mathbf{X}_{L-1}) \frac{\partial \hat{W}_L}{\partial \varepsilon_L} \mathbf{X}_{L-1}.\end{aligned}$$

So:

$$\begin{aligned}\frac{\partial \mathcal{L}_{\text{MS}}}{\partial \varepsilon_L} &= \text{Re}\left\{ (\mathbf{X}_L - \mathbf{T})^\dagger \, \boldsymbol{\phi}'_L(\hat{W}_L \mathbf{X}_{L-1}) \frac{\partial \hat{W}_L}{\partial \varepsilon_L} \mathbf{X}_{L-1} \right\} = \\ &= \text{Re}\left\{ (\mathbf{X}_L - \mathbf{T})^\dagger \, \boldsymbol{\phi}'_L(\mathbf{Y}_L) \frac{\partial \hat{W}_L}{\partial \varepsilon_L} \mathbf{X}_{L-1} \right\}.\end{aligned} \tag{24}$$

Note that $\boldsymbol{\phi}_L(\hat{W}_L \mathbf{X}_{L-1}) = \boldsymbol{\phi}_L(\mathbf{Y}_L)$, where $\mathbf{Y}_L = \hat{W}_L \mathbf{X}_{L-1}$. Therefore, the derivative of $\boldsymbol{\phi}_L$ with respect to ε_L can be computed using the chain rule as

$$\frac{\partial \boldsymbol{\phi}_L}{\partial \varepsilon_L} = \frac{\partial \boldsymbol{\phi}_L}{\partial \mathbf{Y}_L} \frac{\partial \mathbf{Y}_L}{\partial \varepsilon_L}. \tag{25}$$

Here, $\mathbf{Y}_L$ is treated as a complex argument and the derivative $\partial \mathbf{Y}_L / \partial \varepsilon_L$ captures how the pre-activation vector changes with the local permittivity. Therefore $\boldsymbol{\phi}'_L(\mathbf{Y}_L) = \partial \boldsymbol{\phi}_L / \partial \mathbf{Y}_L$.
We observe that the derivative $\frac{\partial \hat{W}_L}{\partial \varepsilon_L}$ is technically a matrix whose entries are the derivatives of each element of $\hat{W}_L$ with respect to the scalar parameter ε_L. When we multiply it on the right by the vector $\mathbf{X}_{L-1}$ and on the left by the element-wise derivative $\boldsymbol{\phi}'_L(\mathbf{Y}_L)$ (represented as a diagonal matrix) and the conjugate error vector $(\mathbf{X}_L - \mathbf{T})^\dagger$, the result is a scalar. Intermediate multiplications, such as $\boldsymbol{\phi}'_L(\mathbf{Y}_L) \frac{\partial \hat{W}_L}{\partial \varepsilon_L} \mathbf{X}_{L-1}$, yield a vector, but taking the inner product with the conjugate error vector and the real part produces the final scalar derivative of the loss with respect to ε_L (Eq. (24)).
Then:

$$\frac{\partial \mathcal{L}_{\text{MS}}}{\partial \varepsilon_L} = \text{Re}\left\{ [\boldsymbol{\phi}'_L(\mathbf{Y}_L)(\mathbf{X}_L - \mathbf{T})^*]^\top \frac{\partial \hat{W}_L}{\partial \varepsilon_L} \mathbf{X}_{L-1} \right\}, \tag{26}$$

expression that we can compactly rewrite as follows:

$$\frac{\partial \mathcal{L}_{\mathrm{MS}}}{\partial \varepsilon_L} = \mathrm{Re}\left\{ \boldsymbol{\delta}_L^\top \frac{\partial \hat{W}_L}{\partial \varepsilon_L} \mathbf{X}_{L-1} \right\}, \tag{27}$$

where we defined $\boldsymbol{\delta}_L \coloneqq \boldsymbol{\phi}'_L(\mathbf{Y}_L)(\mathbf{X}_L - \mathbf{T})^*$.
Repeating the same procedure for $\frac{\partial \mathcal{L}_{\mathrm{MS}}}{\partial \varepsilon_{L-1}}$, we derive a generalization of equation (27) to an arbitrary layer $\ell \in \{1, 2, ..., L\}$. This leads to the following recursive definition:

$$\boldsymbol{\delta}_\ell = \boldsymbol{\phi}'_\ell(\mathbf{Y}_\ell)\hat{W}_{\ell+1}^\top \boldsymbol{\delta}_{\ell+1}. \tag{28}$$

The generalized expression for (27) is:

$$\frac{\partial \mathcal{L}_{\mathrm{MS}}}{\partial \varepsilon_\ell} = \mathrm{Re}\left\{ \boldsymbol{\delta}_\ell^\top \frac{\partial \hat{W}_\ell}{\partial \varepsilon_\ell} \mathbf{X}_{\ell-1} \right\}. \tag{29}$$

Equations (28) and (29) directly recover the classical backpropagation algorithm. The error signal $\boldsymbol{\delta}_{\ell+1}$, computed at layer $\ell+1$, is propagated backward through the transpose of the linear operator $\hat{W}_{\ell+1}$ and then multiplied with the local activation derivative $\boldsymbol{\phi}'_\ell(\mathbf{Y}_\ell)$. In this way, the error is transported recursively from the output layer toward the input. For the computation of the gradient of the loss with respect to the parameters at layer ℓ, all the remaining quantities in (29), namely $\frac{\partial \hat{W}_\ell}{\partial \varepsilon_\ell}$ and $\mathbf{X}_{\ell-1}$, are retrieved locally at the same layer. Hence, once the backpropagated error $\boldsymbol{\delta}_\ell$ is available, the gradient evaluation relies exclusively on locally accessible physical and optical information.

4.2.3 The adjoint variable method for on-chip gradient computation

The error can be physically measured at the output of the system and, by exploiting the reciprocity of the optical network, it can be propagated backward through the mesh. Moreover, the optical field $\mathbf{X}_{\ell-1}$ is directly accessible by measurement at each layer. The remaining crucial quantity is the evaluation of the derivative $\frac{\partial \hat{W}_\ell}{\partial \varepsilon_\ell}$. Hughes et al. (ref. [11]) address this point showing that this derivative can be obtained as the solution of an adjoint electromagnetic problem, thus enabling a fully in-situ computation of the gradient. To do so, we have to resort to first-principles electrodynamics.

Maxwell and Helmoltz equations

We consider the following Maxwell equations (in time domain):

$$\nabla \times \boldsymbol{E} = -\frac{\partial \boldsymbol{B}}{\partial t} \qquad \nabla \times \boldsymbol{B} = \mu_0 \boldsymbol{J} + \mu_0 \varepsilon_0 \varepsilon_r(\boldsymbol{r}) \frac{\partial \boldsymbol{E}}{\partial t}$$

where $\mathbf{E}$ and $\mathbf{B}$ are respectively the electric and magnetic fields, $\boldsymbol{J}$ the current density, ε_0 and $\varepsilon_r(\boldsymbol{r})$ the vacuum and relative dielectric constants (permittivities), μ_0 the vacuum permeability. Our assumptions are:

- *Linear, isotropic, non-dispersive medium* (electric displacement field formula):

$$\boldsymbol{D} = \varepsilon_0 \varepsilon_r(\boldsymbol{r}) \boldsymbol{E}$$

- *Non-ferromagnetic material* (magnetic field intensity formula):

$$\boldsymbol{B} = \mu_0 \boldsymbol{H}$$

- *Time-Harmonic ansatz:* fields oscillate sinusoidally at frequency ω:

$$\boldsymbol{E}(\boldsymbol{r}, t) = \boldsymbol{E}(\boldsymbol{r}) e^{-j\omega t} \qquad \Rightarrow \qquad \frac{\partial}{\partial t} \to -j\omega$$

This transforms Maxwell equations to frequency domain:

$$\begin{aligned} \nabla \times \boldsymbol{E} &= j\omega \boldsymbol{B} \\ \nabla \times \boldsymbol{B} &= \mu_0 [\boldsymbol{J} - j\omega \varepsilon_0 \varepsilon_r(\boldsymbol{r}) \boldsymbol{E}] \end{aligned}$$

Taking the curl of the first equation and substituting the second:

$$\nabla \times (\nabla \times \boldsymbol{E}) = j\omega \nabla \times \boldsymbol{B} = j\omega \mu_0 [\boldsymbol{J} - j\omega \varepsilon_0 \varepsilon_r(\boldsymbol{r}) \boldsymbol{E}]$$

With $k_0 = \omega \sqrt{\mu_0 \varepsilon_0}$ (vacuum wavenumber), we obtain:

$$[\nabla \times \nabla \times (\cdot) - k_0^2 \varepsilon_r(\boldsymbol{r})] \boldsymbol{E} = j\omega \mu_0 \boldsymbol{J} \tag{30}$$

This is the **Helmholtz equation** (curl-curl formulation) for the electric field in frequency domain, which can be rewritten in operator form as:

$$\hat{A}[\varepsilon_r(\boldsymbol{r})]\, \mathbf{E} = j\omega \mu_0 \boldsymbol{J} \tag{31}$$

where $\hat{A}[\varepsilon_r(\boldsymbol{r})] = \nabla \times \nabla \times (\cdot) - k_0^2 \varepsilon_r(\boldsymbol{r})$ is a linear differential operator acting on the electric field, which encodes the medium reaction to the activation of the source.

We can model the interaction of a material with an electromagnetic source with eq. (31), which is a compact form of the Helmoltz equation. All the material information is stored inside the differential operator $\hat{A}$, that depends on its permittivity spatial distribution. This material hosts the generated electric field that satisfies the equality. The light propagating in our neural network can be described using this equation.

Consider the ℓ-th *OIU* inside the network. We say that the amplitude of the electric field generated inside the *OIU* is as high as the propagating field matches the characteristics of the guidable mode. We model this mathematically introducing the projection operators $\hat{P}_{in}$ and $\hat{P}_{out}$, so that [10]:

$$\mathbf{X}_{in} = \hat{P}_{in}\,\mathbf{e}\,, \quad \mathbf{Y}_{out} = \hat{P}_{out}\,\mathbf{e}, \tag{32}$$

where $\mathbf{e}$ is the total electric field propagating at layer ℓ. This field is the sum over the electric fields hosted at each input and output port. With this notation, eq. (18) becomes:

$$\hat{W}\,\hat{P}_{in}\,\mathbf{e} = \hat{P}_{out}\,\mathbf{e}. \tag{33}$$

Taking the derivative with respect to ε (that is one of the network's parameters at layer ℓ):

$$\frac{\partial \hat{W}}{\partial \varepsilon}\,\hat{P}_{in}\,\mathbf{e} + \hat{W}\frac{\partial(\hat{P}_{in}\,\mathbf{e})}{\partial \varepsilon} = \frac{\partial(\hat{P}_{out}\,\mathbf{e})}{\partial \varepsilon}$$

Since the input vector $\hat{P}_{in}\,\mathbf{e}$ is independent of ε, the second term is zero. Hence:

$$\frac{\partial \hat{W}}{\partial \varepsilon}\,\hat{P}_{in}\,\mathbf{e} = \frac{\partial(\hat{P}_{out}\,\mathbf{e})}{\partial \varepsilon}.$$

$\hat{P}_{out}$ and $\hat{P}_{in}$ do not depend on ε[11]. However, the total electric field $\mathbf{e}$ depends on ε through the Helmholtz equation applied in the region of space covered by the associated phase shifter. The virtual source that enables us to write this equation is $\boldsymbol{\sigma}$, so it reads $\hat{A}(\varepsilon)\,\mathbf{e} = \boldsymbol{\sigma}$, thus providing us $\mathbf{e} = \hat{A}^{-1}(\varepsilon)\,\boldsymbol{\sigma}$ and:

$$\frac{\partial(\hat{P}_{out}\,\mathbf{e})}{\partial \varepsilon} = \hat{P}_{out}\frac{\partial \mathbf{e}}{\partial \varepsilon} = \hat{P}_{out}\frac{\partial \hat{A}^{-1}(\varepsilon)}{\partial \varepsilon}\boldsymbol{\sigma}.$$

So:

$$\frac{\partial \hat{W}}{\partial \varepsilon}\,\hat{P}_{in}\,\mathbf{e} = -\hat{P}_{out}\,\hat{A}^{-1}(\varepsilon)\frac{\partial \hat{A}(\varepsilon)}{\partial \varepsilon}\hat{A}^{-1}(\varepsilon)\,\boldsymbol{\sigma}.$$

This becomes:

$$\frac{\partial \hat{W}_\ell}{\partial \varepsilon_\ell}\,\hat{P}^{\ell}_{in}\,\mathbf{e}_\ell = -\hat{P}^{\ell}_{out}\,\hat{A}^{-1}(\varepsilon_\ell)\frac{\partial \hat{A}(\varepsilon_\ell)}{\partial \varepsilon_\ell}\hat{A}^{-1}(\varepsilon_\ell)\,\boldsymbol{\sigma}_{\ell-1},$$

where $\boldsymbol{\sigma}_{\ell-1}$ is the virtual source term that creates the input field amplitudes $\mathbf{X}_{\ell-1}$ at the input ports [12]. Therefore, $\mathbf{e}_\ell$ is the forward electric field dis-

[10] The projection operators $\hat{P}_{in}$ and $\hat{P}_{out}$ extract modal amplitudes from the total field $\mathbf{e}$ by projecting it onto the respective mode profiles.

[11] The guided mode profile is determined by the waveguide geometry and by the material permittivity distribution, while the phase shifter acts only as a perturbative element that modifies the propagation phase of an already existing guided mode. We can therefore say that the projection operators do not depend on the permittivities of the phase shifters.

[12] $\mathbf{X}_{\ell-1} = \hat{P}^{\ell}_{in}\,\mathbf{e}_\ell$.

tribution within the ℓ-th *OIU*. The gradient of the loss with respect to ε_ℓ becomes:

$$\frac{\partial \mathcal{L}_{MS}}{\partial \varepsilon_\ell} = -\mathrm{Re}\Big\{ \boldsymbol{\delta}_\ell^\top \, \hat{P}_{out}^\ell \, \hat{A}_\ell^{-1} \frac{\partial \hat{A}_\ell}{\partial \varepsilon_\ell} \hat{A}_\ell^{-1} \boldsymbol{\sigma}_{\ell-1} \Big\}. \tag{34}$$

In our notation, the virtual source term generating the entering field inside the ℓ-th layer is denoted as $\boldsymbol{\sigma}_{\ell-1}$. Therefore, this source term generates what we now call the forward field $\mathbf{e}_\ell^{\rightarrow} \equiv \mathbf{e}_\ell = \hat{A}_\ell^{-1} \boldsymbol{\sigma}_{\ell-1}$.
We notice that the projection matrix of an electric field onto a guided mode is symmetric in a linear, lossless and isotropic waveguide. Physically, this makes sense because the material properties are the same whether the field enters from the left or from the right. In other words, the coupling from left to right equals the coupling from right to left. Hence, the projection matrix symmetric. Then: $(\hat{P}_{out}^\ell)^\top \boldsymbol{\delta}_\ell = \hat{P}_{out}^\ell \boldsymbol{\delta}_\ell$. Moreover, since for any projection matrix $\hat{P}^2 = \hat{P}$, then $\hat{P}_{out}^\ell \boldsymbol{\delta}_\ell = \boldsymbol{\delta}_\ell$, as vectors like $\boldsymbol{\delta}_\ell$ are already optical amplitudes and so the result of the projection of a physically injected field over the guidable modes. In eq. (34), $\hat{A}_\ell^{-1} \boldsymbol{\sigma}_{\ell-1}$ represents the total forward field distribution propagated through the layer. We can impose that $\boldsymbol{\delta}_\ell^\top \hat{A}_\ell^{-1} = (\mathbf{e}_\ell^{\leftarrow})^\top$, i.e the backpropagated field from the output ports[13]. We can rewrite eq. (34):

$$\frac{\partial \mathcal{L}_{MS}}{\partial \varepsilon_\ell} = -\mathrm{Re}\Big\{ [\mathbf{e}_\ell^{\leftarrow}]^\top \frac{\partial \hat{A}_\ell}{\partial \varepsilon_\ell} \mathbf{e}_\ell^{\rightarrow} \Big\}. \tag{35}$$

Using the operator expression in eq. (30), we compute $\frac{\partial \hat{A}_\ell}{\partial \varepsilon_\ell}$.
We know that $\hat{A}_\ell$ denote the continuous Helmholtz operator restricted to the ℓ-th OIU; after spatial discretization we approximate it by a discrete matrix $\hat{A}_\ell^h$, where h is the characteristic discretization parameter (grid spacing), i.e., $\hat{A}_\ell^h$ acting on the vector of discrete field samples. We consider $\mathbf{r}_{ps}^\ell$ as the set of spatial grid points spanned by the phase shifter with permittivity ε_ℓ inside the ℓ-th OIU. After spatial discretization, each point $\mathbf{r}_i^\ell$ is uniquely associated with a row (or column) index i. Since the permittivity variation affects only the grid points belonging to the phase shifter region, the entries of the derivative matrix are given by:

$$\left(\frac{\partial \hat{A}_\ell^h}{\partial \varepsilon_\ell} \right)_{ij} = \begin{cases} -\, k_0^2 \, \delta_{ij}, & \mathbf{r}_i^\ell \in \mathbf{r}_{ps}^\ell, \\ 0, & \mathbf{r}_i^\ell \notin \mathbf{r}_{ps}^\ell, \end{cases} \tag{36}$$

where δ_{ij} denotes the Kronecker delta[14] [14]. Substituting this expression into the gradient formula, the derivative of the mean-square loss with respect to

[13]This makes sense if you treat the error signal injected from the output ports as a source term for the backpropagating field.

[14]The Kronecker delta appears because, in the frequency-domain Helmholtz operator, the

the permittivity ε_ℓ simplifies to:

$$\frac{\partial \mathcal{L}_{MS}}{\partial \varepsilon_\ell} = k_0^2 \,\mathrm{Re}\left\{ \sum_{\mathbf{r}^\ell \in \mathbf{r}^\ell_{ps}} \left[\mathbf{e}^{\leftarrow}_\ell(\mathbf{r}^\ell)\right]^\top \mathbf{e}^{\rightarrow}_\ell(\mathbf{r}^\ell) \right\}. \tag{37}$$

The gradient of the loss admits a clear physical interpretation: it is (up to known constants) a *local overlap* between the forward field and an adjoint field. In practice, we first perform a standard forward propagation through the OIU to obtain the field throughout the device. At the output, the mismatch with the target produces an error signal, from which we construct the adjoint sources; these sources are then backpropagated through the *same* structure. Inside each phase shifter, the pointwise overlap between the forward and adjoint fields yields the sensitivity of the loss to local permittivity changes. Intuitively, the spatial locations that contribute most to the loss are those where the two fields simultaneously have large magnitude: where their overlap is strongest, a small perturbation produces the largest change in the objective, so these points dominate the gradient.
This adjoint formulation is advantageous because it avoids the computationally-expensive alternative of computing sensitivities parameter-by-parameter. A naive approach (e.g. finite differences) would require one additional full simulation per phase-shifter parameter, leading to a cost that scales as $O(p)$ forward solves for p parameters. In contrast, the adjoint method provides *all* gradients with essentially the cost of two propagations: one forward and one adjoint (backward), plus the cost of reading out the output to form the error and constructing the adjoint sources from that. Since the dominant cost is wave propagation, the overall complexity becomes effectively independent of the number of parameters, making large-scale optimization tractable.

4.2.4 Measuring the overlap between the forward and adjoint fields

The complex conjugate operation, in the frequency domain, as we can see from the time-harmonic ansatz that we have chosen for the electric field, corresponds to its time reversal (see box 4.2.3).
Interfering $\mathbf{e}^{\rightarrow}_\ell$ with the time-reversed adjoint $[\mathbf{e}^{\leftarrow}_\ell]^*$ yields the local intensity:

$$I^\ell = \left|\mathbf{e}^{\rightarrow}_\ell + (\mathbf{e}^{\leftarrow}_\ell)^*\right|^2 = \left|\mathbf{e}^{\rightarrow}_\ell\right|^2 + \left|\mathbf{e}^{\leftarrow}_\ell\right|^2 + 2\,\mathrm{Re}\left\{\mathbf{e}^{\rightarrow}_\ell \cdot \mathbf{e}^{\leftarrow}_\ell\right\},$$

permittivity enters as a pointwise (multiplicative) term $-k_0^2\,\varepsilon(\mathbf{r})$. Upon discretization on nodal/grid degrees of freedom, this contribution becomes diagonal (local), so differentiating with respect to ε_ℓ produces a diagonal matrix supported only on the phase-shifter nodes, hence the δ_{ij}.

Thus:

$$2\ \mathrm{Re}\{\mathbf{e}_\ell^{\rightarrow} \cdot \mathbf{e}_\ell^{\leftarrow}\} = I_\ell - I_\ell^{\rightarrow} - I_\ell^{\leftarrow},$$

with $I_\ell^{\rightarrow} = \left|\mathbf{e}_\ell^{\rightarrow}\right|^2$ and $I_\ell^{\leftarrow} = \left|\mathbf{e}_\ell^{\leftarrow}\right|^2$ measured separately. Therefore, if we can make the two fields $\mathbf{e}_\ell^{\rightarrow}$ and $[\mathbf{e}_\ell^{\leftarrow}]^*$ interfere inside the device, the needed cross term appears in an intensity measurement.
Then, we have to search for an input optical vector, namely $\mathbf{Z}_{\ell-1}$, so that at the output of the OIU we have $\boldsymbol{\delta}_\ell^*$, as $(\mathbf{e}_\ell^{\leftarrow})^* = \hat{A}_\ell^{-1}\boldsymbol{\delta}_\ell^*$. For our lossless reciprocal system we impose that:

$$\mathbf{Z}_{\ell-1}^* \equiv \hat{W}_\ell^T\, \boldsymbol{\delta}_\ell.$$

Applying the complex conjugate operation to both sides of the equation:

$$\mathbf{Z}_{\ell-1} \equiv \hat{W}_\ell^\dagger\, \boldsymbol{\delta}_\ell^* = \hat{W}_\ell^{-1}\, \boldsymbol{\delta}_\ell^*.$$

Since we know that, from eq. (18):

$$\mathbf{Y}_\ell = \hat{W}_\ell\, \mathbf{Z}_{\ell-1} \implies \mathbf{Y}_\ell = \boldsymbol{\delta}_\ell^*.$$

we can say that launching $\mathbf{Z}_{\ell-1}$ at the *input* produces at the *output* the field $\boldsymbol{\delta}_\ell^*$, i.e. a forward-propagating copy of the time-reversed adjoint source. Experimentally, $\mathbf{Z}_{\ell-1}$ can be obtained by injecting $\boldsymbol{\delta}_\ell$ from the output ports, measuring the resulting fields at the input ports and taking the complex conjugate.

Experimental Procedure for Gradient Measurement

1. **Forward pass**
   Launch the forward input $\mathbf{X}_{\ell-1}$ at the *input* ports and measure, at each phase shifter, the forward-field intensity
   $$I_\ell^{\rightarrow} = \left|\mathbf{e}_\ell^{\rightarrow}\right|^2.$$

2. **Adjoint pass**
   Inject the adjoint sources $\boldsymbol{\delta}_\ell$ at the *output* ports and measure, at each phase shifter, the backward (adjoint) intensity
   $$I_\ell^{\leftarrow} = \left|\mathbf{e}_\ell^{\leftarrow}\right|^2.$$

3. **Time-reversed adjoint at the input**
   Send $\boldsymbol{\delta}_\ell$ from the output ports, measure the resulting complex fields at the input ports and take the complex conjugate to obtain the time-reversed adjoint input:
   $$\mathbf{Z}_{\ell-1}^* = \widehat{W}_\ell^T \boldsymbol{\delta}_\ell, \qquad \mathbf{Z}_{\ell-1} = \widehat{W}_\ell^{-1} \boldsymbol{\delta}_\ell^*.$$

4. **Interference measurement**
   Simultaneously launch $\mathbf{X}_{\ell-1}$ and $\mathbf{X}_{\mathrm{TR},\ell-1}$ at the input ports. Record, at each phase shifter, the total intensity
   $$I_\ell = \left|\mathbf{e}_\ell^{\rightarrow} + [\mathbf{e}_\ell^{\leftarrow}]^*\right|^2.$$

5. **Gradient retrieval**
   Compute the gradient from the three intensity measurements:
   $$\frac{\partial \mathcal{L}}{\partial \varepsilon_\ell} = \frac{k_0^2}{2}(I_\ell - I_\ell^{\rightarrow} - I_\ell^{\leftarrow}).$$

This algorithm is shown in Fig. 5.

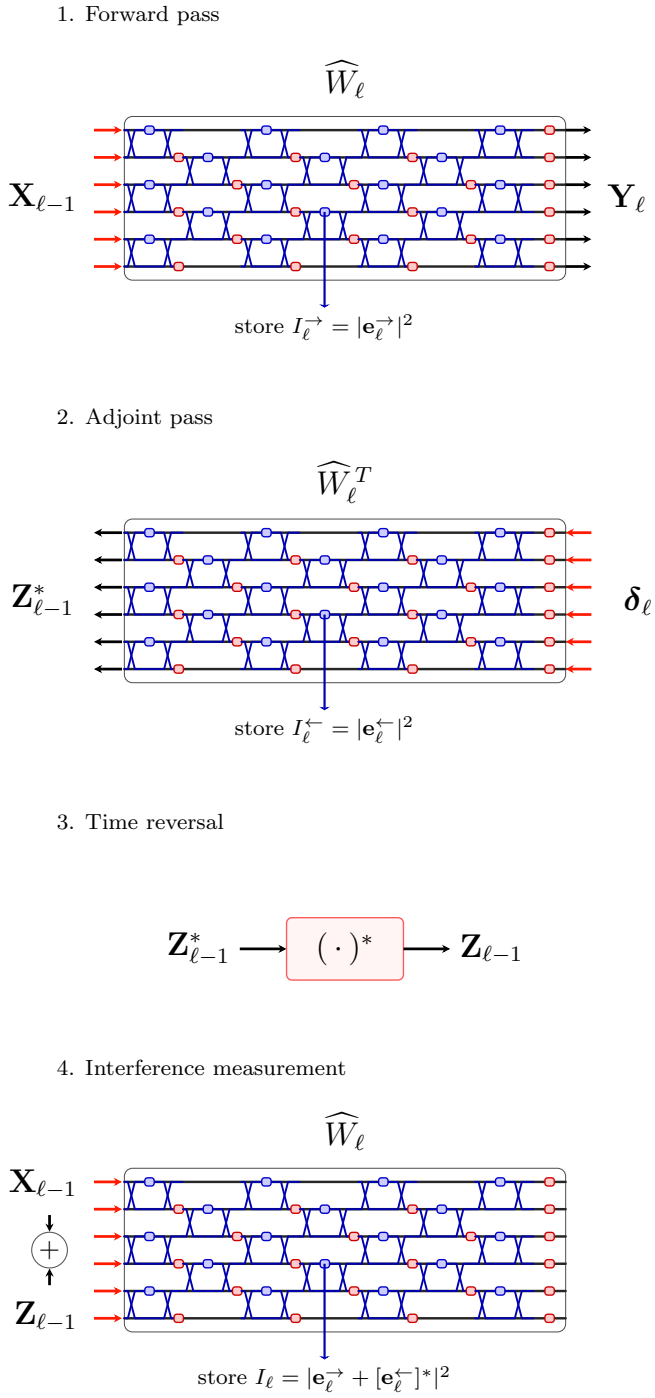

Figure 5: Graphical representation of the in-situ training algorithm. The gradient of the loss function with respect to the phase shifter parameter ε_ℓ is computed from the intensity difference: $\frac{\partial \mathcal{L}}{\partial \varepsilon_\ell} = \frac{k_0^2}{2}\left(I_\ell - I_\ell^{\rightarrow} - I_\ell^{\leftarrow}\right)$.

5 Physics-informed Photonic Neural Networks

Suppose to work in an unsupervised framework. Let the function $\mathcal{L}$ be a real-valued functional of a complex vector $\mathbf{X}$. Its infinitesimal first-order variation can be written as:

$$\delta\mathcal{L} = \mathrm{Re}\left\{ \left(\frac{\partial \mathcal{L}}{\partial \mathbf{X}} \right)^{\dagger} \delta\mathbf{X} \right\}, \tag{38}$$

which is the complex counterpart of the real expression $\delta\mathcal{L} = (\nabla\mathcal{L})^{\top}\delta\mathbf{x}$. Note that the real part is required in eq. (38) because $\mathcal{L}$ is a real quantity.
For our photonic version of the Deep Ritz Method, we will consider the loss function $\mathcal{L}_{DR}$. We note that, to exploit the photonic circuit presented in the previous section, the cost function $\mathcal{L}_{DR}$ has the same characteristics as $\mathcal{L}$: it is a real-valued function that takes complex vectors as input. The output of our circuit is $\mathbf{X}_L = \boldsymbol{\phi}_L(\mathbf{Y}_L)$, where $\boldsymbol{\phi}_L$ denotes the nonlinear element-wise activation of the last layer, whose pre-activation field is $\mathbf{Y}_L$. Then:

$$\delta\mathbf{X}_L = \boldsymbol{\phi}'_L(\mathbf{Y}_L)\,\delta\mathbf{Y}_L.$$

Substituting into eq. (38) yields:

$$\delta\mathcal{L}_{DR} = \mathrm{Re}\left\{ \left(\frac{\partial \mathcal{L}_{DR}}{\partial \mathbf{X}_L} \right)^{\dagger} \left(\boldsymbol{\phi}'_L(\mathbf{Y}_L)\,\delta\mathbf{Y}_L \right) \right\}.$$

From this, using the chain rule, we obtain:

$$\frac{\partial \mathcal{L}_{DR}}{\partial \varepsilon_L} = \mathrm{Re}\left\{ \left[\boldsymbol{\phi}'_L(\mathbf{Y}_L) \left(\frac{\partial \mathcal{L}_{DR}}{\partial \mathbf{X}_L} \right)^{*} \right]^{\top} \frac{\partial \hat{W}_L}{\partial \varepsilon_L}\, \mathbf{X}_{L-1} \right\}. \tag{39}$$

Therefore:

$$\boldsymbol{\delta}_L = \boldsymbol{\phi}'_L(\mathbf{Y}_L) \left(\frac{\partial \mathcal{L}_{DR}}{\partial \mathbf{X}_L} \right)^{*}.$$

Physically, this is the quantity that must be injected into the adjoint optical system so that the measured overlap between forward and adjoint fields produces the correct gradient with respect to the physical parameters. The result we have found is consistent with the method proposed in ref. [11] and enables us to generalize the photonic training algorithm to any loss function. In particular, this result is fundamental for extending our work to unsupervised learning contexts, since the expression above allows us to avoid dependence on targets, which are usually present in supervised learning tasks. In particular, the quantity $\frac{\partial \mathcal{L}_{DR}}{\partial \mathbf{X}_L}$ must be computed from our specific loss function in order to adapt the photonic training algorithm to our case. We will use the scheme depicted in Fig. 6.

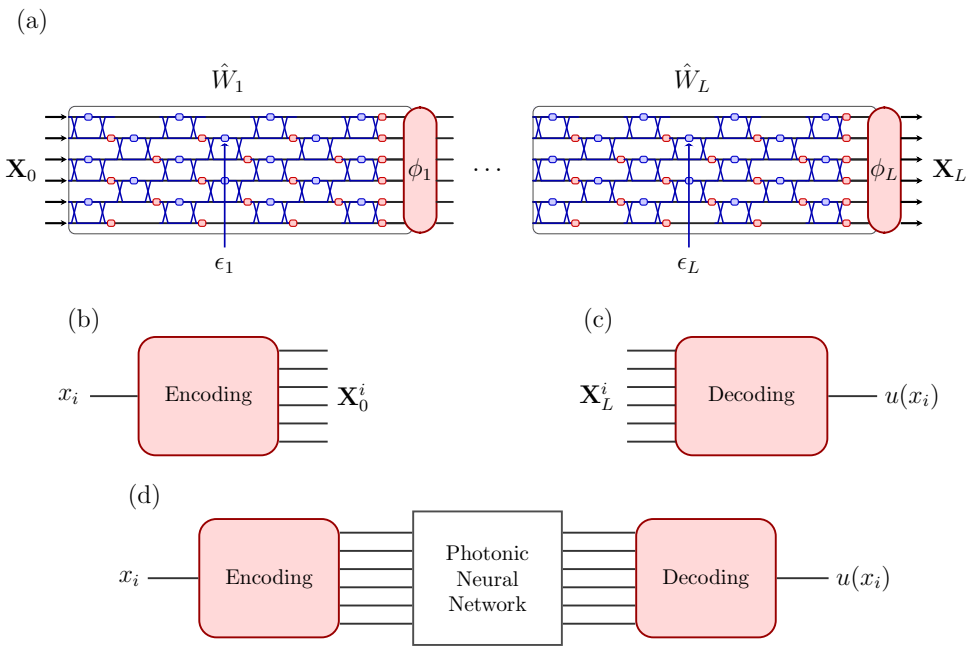

Figure 6: Schematic representation of the photonic neural network architecture. (a) A cascaded photonic neural network with L optical interference units (OIUs) and nonlinear activation layers; phase shifters are trainable parameters. (b) Input encoding maps the scalar x_i to a 6-mode state $\mathbf{X}_0^i$. (c) Linear readout (decoding) maps the 6-mode output $\mathbf{X}_L^i$ to the scalar prediction $u(x_i)$. (d) End-to-end pipeline: $x_i \to$ encoding $\to$ photonic neural network $\to$ decoding $\to u(x_i)$.

5.1 The Poisson case

We consider the (1D) Poisson problem on a domain Ω with homogeneous Dirichlet boundary conditions:

$$-\Delta u(x) = f(x), \qquad x \in \Omega, \qquad u|_{\partial\Omega} = 0.$$

The Deep Ritz variational functional to minimize is

$$\mathcal{L}_{DR}[\theta] \;=\; \int_\Omega \left(\tfrac{1}{2}|\nabla u_\theta(x)|^2 - f(x)\, u_\theta(x)\right)\, dx.$$

Discrete quadrature

Let $\{x_i\}_{i=1}^Q \subset \Omega$ be the quadrature points with weights $w_i > 0$. We approximate the functional by a discrete sum:

$$\mathcal{L}_{DR}(\theta) \approx \sum_{i=1}^{Q} w_i\, g_i, \qquad g_i \coloneqq \tfrac{1}{2}\left|\nabla u_\theta(x_i)\right|^2 - f(x_i)\, u_\theta(x_i).$$

Assume the photonic network (last layer) produces a vector-valued complex field $\mathbf{X}_L^i$ when fed with input $\mathbf{X}_0^i$ corresponding to x_i. We introduce a linear readout that maps $\mathbf{X}_L^i$ to the scalar $u_\theta(x_i)$. A linear readout is described by the following:

$$u_\theta(x_i) \;=\; \mathrm{Re}\{\mathbf{a}^\dagger \mathbf{X}_L^i\} + b,$$

where $\mathbf{a} \in \mathbb{C}^n$ and $b \in \mathbb{R}$ are readout parameters (they can be fixed or trained). Taking the real part ensures $u_\theta(x_i) \in \mathbb{R}$ as required by the PDE.
For Poisson, the pointwise contribution g_i depends on $u_\theta(x_i)$ and on its spatial derivatives. The functional derivative of g_i with respect to the scalar $u_\theta(x_i)$ is:

$$\frac{\partial g_i}{\partial u_\theta(x_i)} = -f(x_i) \;-\; \Delta u_\theta(x_i),$$

that is, the PDE residual evaluated at x_i.
By the chain rule, the derivative of the total discrete loss with respect to the complex vector $\mathbf{X}_L^i$ is

$$\frac{\partial \mathcal{L}_{DR}}{\partial \mathbf{X}_L^i} = \sum_{j=1}^{Q} w_j \frac{\partial g_j}{\partial u_\theta(x_j)} \frac{\partial u_\theta(x_j)}{\partial \mathbf{X}_L^i}.$$

Since $\mathbf{X}_L^j$ depends only on x_j , the Jacobian $\partial u_\theta(x_j)/\partial \mathbf{X}_L^i$ vanishes for $i \neq j$. Hence the sum reduces to a single term:

$$\frac{\partial \mathcal{L}_{DR}}{\partial \mathbf{X}_L^i} = w_i \left(\frac{\partial g_i}{\partial u_\theta(x_i)}\right) \frac{\partial u_\theta(x_i)}{\partial \mathbf{X}_L^i}.$$

With the readout $u_\theta(x_i) = \mathrm{Re}\{\mathbf{a}^\dagger \mathbf{X}_L(x_i)\} + b$ we obtain

$$\frac{\partial u_\theta(x_i)}{\partial \mathbf{X}_L^i} = \frac{\partial}{\partial \mathbf{X}_L} \mathrm{Re}\{\mathbf{a}^\dagger \mathbf{X}_L\} = \frac{1}{2}\,\mathbf{a}^*,$$

where $(\cdot)^*$ denotes complex conjugation and the factor $1/2$ arises from $\mathrm{Re}(z) = \frac{1}{2}(z + z^*)$. Therefore

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}_L^i} \;=\; w_i \,\big(- f(x_i) - \Delta u_\theta(x_i)\big)\, \frac{1}{2}\,\mathbf{a}^* \,. \tag{40}$$

We notice that the network stops learning when the strong residual ofthe equation approaches zero.

Constructing the adjoint source $\boldsymbol{\delta}_L$

Recall from the complex backpropagation derivation that the local error vector to be injected as optical adjoint sources is

$$\boldsymbol{\delta}_L(x_i) \;=\; \boldsymbol{\phi}_L'\big(\mathbf{Y}_L(x_i)\big) \left(\frac{\partial \mathcal{L}}{\partial \mathbf{X}_L(x_i)} \right)^*,$$

where $\boldsymbol{\phi}_L'$ denotes the element-wise derivative of the last-layer activation with respect to the pre-activation $\mathbf{Y}_L$, and $(\cdot)^*$ is complex conjugation. Using (40) we obtain the practical form:

$$\boldsymbol{\delta}_L(x_i) \;=\; \frac{w_i}{2}\,\big(- f(x_i) - \Delta u_\theta(x_i)\big)\; \boldsymbol{\phi}_L'\big(\mathbf{Y}_L(x_i)\big)\,\mathbf{a}. \tag{41}$$

This $\boldsymbol{\delta}_L(x_i)$ is the complex-valued vector that must be synthesized and injected from the output ports according to the procedure described in ref. [11]. Notice that, in general, activation functions are complex-valued. A rigorous treatment of non-holomorphic activations is beyond the scope of this work.

Practical remarks

- **Encoding and Decoding blocks**
  Photonic amplitudes and numerical values of u_θ live in different unit systems. A linear calibration (scale and offset) between measured optical amplitudes and the numerical u_θ must be established prior to reinjecting $\boldsymbol{\delta}_L$.

- **Batching: mini-batch gradient descent**
  In practice one computes $\boldsymbol{\delta}_L$ for a batch of sample points $\{x_i\}_{i\in\mathcal{B}}$ and then synthesizes a composite adjoint source (or processes samples sequentially). Batching reduces variance and makes the procedure efficient.

- **Laplacian's computation**
  If the photonic ansatz is differentiable in x analytically, one may compute Δu_θ via the analytic derivative of the readout; otherwise finite differences across nearby sample points are a robust option (at the cost of extra forward measurements).

- **Workflow with finite differences**
  A pragmatic is: (i) perform forward photonic measurements to obtain $\mathbf{X}_L(x_i)$ and hence $u_\theta(x_i)$ and neighbor values (in general we require a neighbourhood of $u_\theta(x_i)$ because with finite differences the Laplacian computation is translated into a convolution); (ii) compute the strong residual and $\boldsymbol{\delta}_L(x_i)$ numerically off-chip; (iii) reinject the adjoint optical field corresponding to $\boldsymbol{\delta}_L$ and measure overlaps to recover parameter gradients in situ.

- **Workflow in the analytic case**
  The training loop can be fully analytic at the output level: (i) evaluate $u_\theta(x_i)$ and its spatial derivatives analytically; (ii) compute the residual and the output-layer adjoint $\boldsymbol{\delta}_L(x_i)$ in closed form; (iii) inject the corresponding adjoint optical field to directly measure gradient overlaps on-chip. In this setting, no numerical differentiation or finite-difference approximation is required, avoiding the propagation of finite-differences approximation error. Moreover, convolutions, to be closer to the approximated operator, require a lot computational resources, as demonstrated with GPUs usage in literature. The analytic computation of spatial derivatives increases the usage efficiency of our architecture as a propagating recursive scheme is employed.

- **Spatial gradient computation**
  The same hold for the spatial first derivative of $u_\theta(x_i)$. In particular for elliptic problems (and also parabolic ones), as we have showed in the first section that integration by parts lowers the order of the differential operator, weakening the requirement on the solutions' regularity, we will always need the first order spatial derivative of the solution (as the cited ones are second-order differential problems). Therefore, we need the gradient of the solution as a piece for building the loss function, as illustrated in Fig. 2.

Our major contribution relies in the following pipeline for the solution of differential problems driven by photonic hardware.

5.1.1 Photonic Deep Ritz Algorithm

1. Sample a batch $\{x_i\}_{i=1}^{B}$ of size B from a user-defined spatial distribution[15].

2. For each x_i: encode x_i according to the chosen input rule, measure $\mathbf{X}_L(x_i)$, and compute

$$u_\theta(x_i) = \mathrm{Re}\{\mathbf{a}^\dagger \mathbf{X}_L(x_i)\} + b.$$

3. Compute the required spatial derivatives (e.g. $\nabla u_\theta(x_i)$ and, when needed, $\Delta u_\theta(x_i)$) using the analytic approach.

4. Evaluate the loss as the average over the batch points (Monte Carlo estimate).

5. If a stopping criterion is met (maximum number of epochs or tolerance achieved), go to step 9; otherwise, continue.

6. Compute residuals r_i and construct

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}_L^*(x_i)}$$

using eq. (40), then build $\boldsymbol{\delta}_L(x_i)$ (eq. (41)).

7. Synthesize and inject the adjoint fields corresponding to $\boldsymbol{\delta}_L$, measure overlaps $\langle \mathbf{e}^{\rightarrow}, \mathbf{e}^{\leftarrow} \rangle$ as done in ref.[11] and recover $\partial \mathcal{L}/\partial \varepsilon$ for the physical parameters.

8. Update parameters using an optimizer (e.g. Adam) and repeat from step 1.

9. End.

5.2 Analytic computation of spatial derivatives

The network propagates $\mathbf{X}_0(x)$ through photonic layers to produce the output field:

$$\mathbf{X}_L(x) = \hat{\mathcal{N}}_{\mathrm{opt}}(\mathbf{X}_0(x)).$$

[15] We use Monte Carlo sampling and define an epoch as a single training iteration over the current batch. The epoch definition is necessary since a pass over all spatial points is not possible. Updates are performed using the loss averaged over the batch points.

Here $\hat{\mathcal{N}}_{\text{opt}}$ represents the nonlinear transformations performed by the photonic ansatz. The scalar output used for the PDE is obtained via a linear readout:

$$u_\theta(x) = \text{Re}\{\mathbf{a}^\dagger \mathbf{X}_L(x)\} + b.$$

The gradient of the readout with respect to x is obtained using the chain rule:

$$\frac{du_\theta}{dx} = \text{Re}\left\{\mathbf{a}^\dagger \frac{d\mathbf{X}_L}{dx}\right\} = \text{Re}\left\{\mathbf{a}^\dagger \frac{d\mathbf{X}_L}{d\mathbf{X}_0}\frac{d\mathbf{X}_0}{dx}\right\}.$$

We define the *sensitivity* of the network with respect to the input as the following Jacobian operator:

$$\hat{J}_{opt} = \frac{d\mathbf{X}_L}{d\mathbf{X}_0} \in \mathbb{C}^{n_L \times n_0},$$

where n_0 is the input dimension and n_L the number of modes in the last layer. We obtain the following expression:

$$\frac{du_\theta}{dx} = \text{Re}\left\{\mathbf{a}^\dagger \hat{J}_{opt}\frac{d\mathbf{X}_0}{dx}\right\}. \tag{42}$$

Following the same reasoning, we can compute the Laplacian of the readout:

$$\frac{d^2u_\theta}{dx^2} = \text{Re}\left\{\mathbf{a}^\dagger \frac{d^2\mathbf{X}_L}{dx^2}\right\}.$$

5.2.1 Computing the Neural Network's Jacobian

Now consider our network with nonlinear activations between layers:

$$\mathbf{X}_\ell = \phi_\ell(\mathbf{W}_\ell \mathbf{X}_{\ell-1}).$$

Applying the chain rule, the first derivative of the output field at layer ℓ with respect to output field at layer $\ell - 1$ (local sensitivity between two consecutive layers) is:

$$\hat{J}_\ell = \frac{\partial \mathbf{X}_\ell}{\partial \mathbf{X}_{\ell-1}} = \phi'_\ell(\mathbf{W}_\ell \mathbf{X}_{\ell-1})\,\mathbf{W}_\ell, \tag{43}$$

that is, namely, the local Jacobian matrix $\hat{J}_\ell$. By applying the chain rule recursively from the output layer L back to the input layer 0, we obtain the total Jacobian of the neural network. This represents the global sensitivity of the output $\mathbf{X}_L$ with respect to the input vector $\mathbf{X}_0$:

$$\hat{J}_{opt} = \frac{\partial \mathbf{X}_L}{\partial \mathbf{X}_0} = \frac{\partial \mathbf{X}_L}{\partial \mathbf{X}_{L-1}}\frac{\partial \mathbf{X}_{L-1}}{\partial \mathbf{X}_{L-2}}\cdots\frac{\partial \mathbf{X}_1}{\partial \mathbf{X}_0} = \prod_{\ell=L}^{1} \hat{J}_\ell \tag{44}$$

We observe that:

$$\frac{d\mathbf{X}_\ell}{dx} = \phi'_\ell(\mathbf{W}_\ell \mathbf{X}_{\ell-1})\mathbf{W}_\ell \frac{d\mathbf{X}_{\ell-1}}{dx}. \tag{45}$$

Similarly, the second derivative at layer ℓ is

$$\frac{d^2\mathbf{X}_\ell}{dx^2} = \phi''_\ell(\mathbf{W}_\ell \mathbf{X}_{\ell-1})\big(\mathbf{W}_\ell \frac{d\mathbf{X}_{\ell-1}}{dx}\big)^2 + \phi'_\ell(\mathbf{W}_\ell \mathbf{X}_{\ell-1})\mathbf{W}_\ell \frac{d^2\mathbf{X}_{\ell-1}}{dx^2}. \tag{46}$$

This shows that the network's expressiveness is strictly dependent on the second-order smoothness of its activation functions since if $\phi''_\ell = 0$ (e.g. for ReLU activations) the information that is useful to compute the Laplacian vanishes [16].

Once the first and second derivatives of the output field with respect to x are computed recursively through the network[17], the pointwise PDE residual can be evaluated at each quadrature point x_i:

$$r_i = -f(x_i) - \frac{d^2 u_\theta(x_i)}{dx^2}. \tag{47}$$

This residual directly provides the local functional derivative of the Deep Ritz loss with respect to the network output at the sample point. This quantity builds the adjoint source to be injected at the output ports of the photonic chip, following the aforementioned procedure.

[16] No new second-order dependence on the spatial variable is generated within the network and the second derivative can only be propagated from the input. As a consequence, if the input encoding has zero second derivative (as in linear encodings), the Laplacian vanishes identically.

[17] Remark: this approach avoids computing the Hessian with respect to the input $\mathbf{X}_0$, which would be extremely costly. Instead, the recursive computation of the derivatives with respect to x is sufficient for evaluating the Laplacian and the PDE residual.

6 Benchmarks

This section wants to show the results of our simulations using the `neuroptica` library, extended to non-supervised settings as the one of our interest. The following test cases for our *Photonic Deep Ritz Algorithm* represent benchmarks that are useful for the validation of the proposed pipeline. The long term goal is faster and more sustainable large scale simulations.

6.1 1D Poisson problems

The following subsections report 1D experiments implemented, including both homogeneous Dirichlet and mixed (Dirichlet–Neumann) boundary conditions.

6.1.1 Homogeneous Dirichlet boundary conditions

We first consider the one-dimensional (1D) Poisson problem with homogeneous Dirichlet boundary conditions. The strong formulation is:

$$\begin{aligned} -u''(x) &= f(x), \qquad x \in (0,1), \\ u(0) &= 0, \\ u(1) &= 0. \end{aligned}$$

The weak (variational) problem is associated with the following Deep Ritz energy:

$$\mathcal{L}_{DR} = \int_0^1 \left(\tfrac{1}{2}[u'(x)]^2 - f(x)\, u(x) \right) dx.$$

In the photonic implementation, Dirichlet conditions can be enforced by adding penalty terms evaluated at boundary samples. We augmented the minibatch loss with a boundary penalty of the form:

$$\mathcal{L}_{bc,d} = \frac{\lambda_{\mathrm{bc}}}{2B_{\mathrm{bc}}} \sum_{j=1}^{N_{\mathrm{bc}}} u(x_j^{\mathrm{bc}})^2, \qquad x_j^{\mathrm{bc}} \in \{0,1\}.$$

Using the readout layer:

$$u_\theta(x) = \mathrm{Re}\{\mathbf{a}^\dagger \mathbf{X}_L(x)\} + b. \qquad \frac{\partial u(x)}{\partial \mathbf{X}_L^*(x)} = \tfrac{1}{2}\mathbf{a},$$

Each boundary sample contributes to the adjoint source as:

$$\frac{\partial \mathcal{L}_{bc,d}}{\partial \mathbf{X}_L^*}(x_j^{\mathrm{bc}}) = \left(\frac{\lambda_{\mathrm{bc}}}{B_{\mathrm{bc}}} u(x_j^{\mathrm{bc}}) \right) \frac{\partial u(x_j^{\mathrm{bc}})}{\partial \mathbf{X}_L^*} = \frac{\lambda_{\mathrm{bc}}}{2B_{\mathrm{bc}}}\, u(x_j^{\mathrm{bc}})\, \mathbf{a}.$$

6.1.2 Neumann boundary condition at $x = 1$ (mixed BCs)

We consider the one-dimensional Poisson problem

$$\begin{aligned} -u''(x) &= f(x), \qquad x \in (0,1), \\ u(0) &= 0, \\ u'(1) &= g, \end{aligned}$$

where a Dirichlet condition is imposed at $x = 0$ and a Neumann condition at $x = 1$.

Weak formulation

Let

$$V := \{v \in H^1(0,1) \ : \ v(0) = 0\}.$$

Multiplying the strong form by a test function $v \in V$ and integrating by parts, we obtain

$$\int_0^1 u'(x)\, v'(x)\, dx = \int_0^1 f(x)\, v(x)\, dx + g\, v(1), \qquad \forall v \in V.$$

The Neumann datum appears naturally as a boundary term in the weak formulation.
This weak formula is associated with the Deep Ritz energy:

$$\mathcal{L}_{DR}(u) = \int_0^1 \left(\tfrac{1}{2}[u'(x)]^2 - f(x)\, u(x)\right)\, dx - g\, u(1).$$

In the Deep Ritz implementation, interior collocation points $x_i \in (0,1)$ are associated with the strong-form residual

$$r(x_i) = -u''(x_i) - f(x_i),$$

which is used to construct the output adjoint source. In this case the energy formulaton requires an additional boundary contribution at $x = 1$, proportional to $u'(1) - g$. With the photonic readout

$$u(x) = \mathrm{Re}\{\mathbf{a}^\dagger \mathbf{X}_L(x)\} + b, \qquad \frac{\partial u(x)}{\partial \mathbf{X}_L^*(x)} = \tfrac{1}{2}\mathbf{a},$$

the Neumann boundary sample produces the (pointwise) adjoint source

$$\frac{\partial \mathcal{L}_{bc,n}}{\partial \mathbf{X}_L^*}(1) = \big(u'(1) - g\big)\, \frac{\partial u(1)}{\partial \mathbf{X}_L^*} = \frac{u'(1) - g}{2}\, \mathbf{a}. \tag{48}$$

In the stochastic minibatch setting used, the adjoint injection implemented additionally includes the usual Monte Carlo normalization associated with averaging over the sampled points. We are basically omitting the explicit first-variation derivation for each boundary-condition type: once stationarity is established by the weak formulation, repeating the algebra case-by-case would obscure the key implementation message. Instead, we highlight an intuitive parallelism: in all our test cases, the adjoint injection output is the gradient of the minibatch loss with respect to the complex conjugate of the output optical amplitude vector $\mathbf{X}_L$ and is always factorized into the same building blocks (consider the loss function as a sum over i-indexed contributions, e.g. interior samples, either Dirichlet or Neumann boundary samples):

$$\frac{\partial \mathcal{L}_i}{\partial \mathbf{X}_L^*} = \underbrace{\frac{1}{N_{\text{batch}}}}_{\text{normalization}} \times \underbrace{\lambda}_{\text{tunable weight}} \times \underbrace{(\text{strong residual})}_{\text{physics mismatch}} \times \underbrace{\frac{1}{2}\mathbf{a}}_{\text{photonic readout}}$$

The global loss is decomposed as:

$$\mathcal{L} = \mathcal{L}_{int} + \mathcal{L}_{bc,d} + \mathcal{L}_{bc,n},$$

where each term contributes to the gradient via the same factorized shape.

Contribution	**Constant term**	**Strong Residual**	**Adjoint Injection**
PDE interior	$\frac{\lambda_{\text{int}}}{N_{int}}$	$-\Delta u - f$	$\frac{\lambda_{int}}{2N_{int}}(-\Delta u - f)\,\mathbf{a}$
Dirichlet BC	$\frac{\lambda_{bc,d}}{N_{bc}}$	$u - h$	$\frac{\lambda_{bc,d}}{2N_{bc}}(u - h)\,\mathbf{a}$
Neumann BC	$\frac{\lambda_{bc,n}}{N_{bc,n}}$	$u' - g$	$\frac{\lambda_{bc,n}}{2N_{bc,n}}(u' - g)\,\mathbf{a}$

Remarks:

- The weights $\lambda_{int}, \lambda_{bc,d}, \lambda_{bc,n}$ provide additional degrees of freedom to balance the relative importance of interior and boundary constraints in the total loss. In practice, these are often chosen so that each term contributes comparably during training.

- For homogeneous Dirichlet conditions ($h = 0$), the strong residual simplifies to just u.
- The Monte–Carlo normalization $1/N_{\text{batch}}$ ensures unbiased gradient estimates when sampling points stochastically.

6.1.3 Notebook setup and analysis

All 1D experiments share the same photonic encoding, readout strategy, and analytic propagation of derivatives. We study the 1D Poisson problem on $\Omega = (0, 1)$ using a simulated N-mode optical mesh network implemented in `neuroptica`, with complex-valued fields.

Photonic encoding and readout
The input embedding injects the physical coordinate on one port, while the remaining ports provide constant bias fields:

$$\mathbf{X}_0(x) = \begin{bmatrix} x \\ 1 \\ \vdots \\ 1 \end{bmatrix} \in \mathbb{C}^N, \qquad x \in (0, 1). \tag{49}$$

Unless otherwise stated, we use $N = 3$ modes. The corresponding input derivatives are

$$\frac{d\mathbf{X}_0}{dx} = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \qquad \frac{d^2\mathbf{X}_0}{dx^2} = \mathbf{0}. \tag{50}$$

This encoding is required because a unitary photonic mesh is intrinsically linear and cannot generate additive offsets internally; constant channels enable affine shifts through interference and the final readout.

The photonic circuit $\mathcal{N}_{opt} : \mathbb{C}^N \to \mathbb{C}^N$ produces

$$\mathbf{X}_L(x) = \mathcal{N}_{opt}(\mathbf{X}_0(x)). \tag{51}$$

A real-valued scalar field is obtained via

$$u_\theta(x) = \mathrm{Re}\big\{\mathbf{a}^\dagger \mathbf{X}_L(x)\big\} + b, \tag{52}$$

where $\mathbf{a} \in \mathbb{C}^N$ and $b \in \mathbb{R}$. In the benchmarks we typically fix $\mathbf{a}$ (e.g. selecting one port) and set $b = 0$.

Photonic architecture
The shallow baseline model is

$$\text{Clements}(N) \;\to\; z^2 \;\to\; \text{Clements}(N) \;\to\; z^2,$$

where z^2 denotes the elementwise holomorphic nonlinearity. For deep benchmarks, we use a depth parameter d corresponding to d repeated blocks of $\text{Clements}(N) \to z^2$.

Analytic derivatives through the network
Derivatives are propagated analytically through the photonic layers.

PDE setting
We consider the 1D Poisson equation

$$-u''(x) = f(x), \qquad x \in (0,1), \tag{53}$$

with either (i) homogeneous Dirichlet conditions $u(0) = u(1) = 0$, or (ii) mixed conditions $u(0) = 0$ and $u'(1) = g$.

6.1.4 1D benchmarks

Common setup (all 1D runs)
We approximate a scalar solution $u_\theta(x)$ on $x \in [0,1]$ with a photonic neural network (simulated via `neuroptica`). The network processes a complex field vector $X_L(x) \in \mathbb{C}^N$, and produces the real-valued readout

$$u_\theta(x) = \text{Re}\{\mathbf{a}^\dagger X_L(x)\} + b.$$

Training is unsupervised: the optimizer minimizes a Deep Ritz (energy) functional and receives a single complex adjoint source term $d\mathcal{L}/dX_L^*$ at the output, which triggers the in-situ Adam update. For each benchmark below we report (i) the solution plot and (ii) the training loss curve.

Benchmark 1: Dirichlet, constant forcing ($f = 1$).

– **PDE and BCs**
We solve the 1D Poisson problem

$$-u''(x) = 1, \qquad x \in (0,1), \qquad u(0) = u(1) = 0,$$

whose closed-form solution is $u_{exact}(x) = \frac{1}{2}x(1-x)$. We minimize the Deep Ritz functional (Dirichlet enforced by penalty)

$$\mathcal{L}_{DR}(u) = \mathbb{E}\big[\tfrac{1}{2}\, u_x(x)^2 - f\, u(x)\big] + \frac{\lambda_{bc,d}}{2}\, \mathbb{E}\big[u(x)^2\big]_{\{x=0,1\}}\,.$$

– **Model**

Shallow photonic model with mesh size $N = 3$ (2 Clements meshes + z^2 nonlinearity blocks, then linear readout).

– **Training hyperparameters**

epochs	batch	step size	$\lambda_{bc,d}$	N
1000	64	0.02	50.0	3

– **Results**

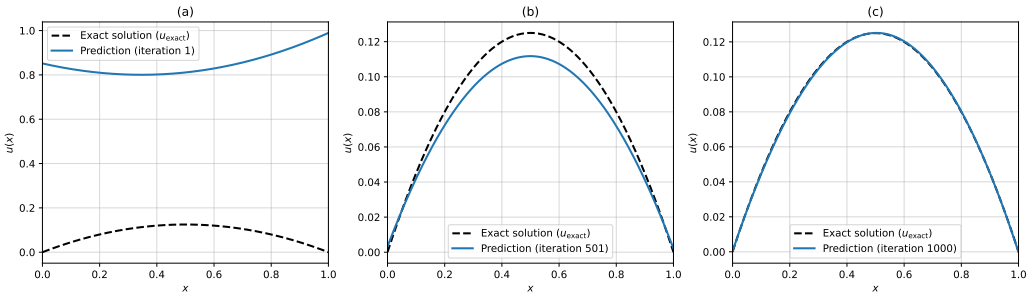

Figure 7: Predicted solution versus the analytical reference.

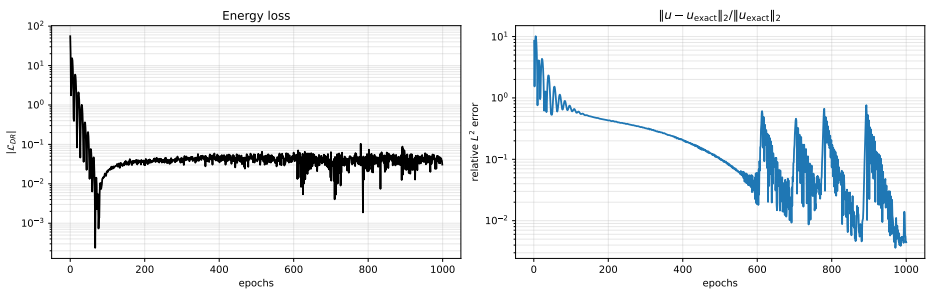

Figure 8: Deep Ritz training loss versus epochs (energy functional) and relative residual $\mathcal{L}^2$ error.

6.1.5 Energy loss interpretation and strong-form residual

Why does a small loss not guarantee an accurate solution?
During training we observed that the Deep Ritz energy loss alone was not sufficiently informative of the pointwise accuracy of the solution. For this reason, we additionally monitored the relative $\mathcal{L}^2$ residual error. Our observations can be summarized as follows:

- The energy loss represents an *average* quantity; large localized errors may have a negligible impact on its value.
- The functional depends mainly on u' (and on u weighted by f), so different functions can exhibit similar energy values while differing significantly pointwise.
- For the Poisson problem with $f = 1$ and exact Dirichlet boundary conditions, the exact solution satisfies

  $$\mathcal{L}_{DR}(u_{exact}) = -\frac{1}{24} \approx -0.0417.$$

  We measured a final loss of -0.0308, which is consistent with the scale of the problem and does not necessarily imply a near-exact solution. Given the quality of the predicted solution, we say that the right energy was found, with a small positive bias mainly due to the presence of the boundary penalty term.

Even when the solution profile appears visually correct, it is useful to inspect the *strong-form residual* evaluated on a dense grid,

$$r(x) = u''(x) + f(x).$$

For the Poisson test considered here, we set $f(x) = 1$, hence

$$r(x) = u''(x) + 1,$$

which should ideally be close to zero in the interior of the domain. We show our result in Fig. 9.
The Deep Ritz method minimizes a *weak* (energy-based) objective and does not explicitly enforce the strong form pointwise. Therefore, non-uniform residuals are expected, especially near the boundaries. Dirichlet conditions are imposed through a penalty term with coefficient $\lambda_{bc,d}$. While this enforces $u(0) \approx u(1) \approx 0$, it does not constrain the curvature u'' at the boundary, often producing a thin boundary layer where the residual becomes large.

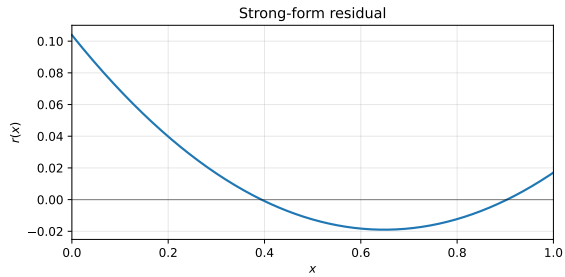

Figure 9: Plotting the strong-form residual of the predicted solution.

It is also important to note that the strong-form residual is *not mathematically required to vanish at the boundary*: the PDE is defined in the interior of the domain, while boundary conditions are imposed separately. Consequently, large residual values near $x = 0$ and $x = 1$ are not necessarily indicative of poor interior PDE satisfaction.
To reduce near/boundary residuals, the following strategies are mainly effective:

1. **Low-frequency bias mitigation**: neural networks trained with gradient-based methods tend to represent low-frequency components first, while high-curvature features (such as boundary layers) are learned more slowly. This spectral bias can delay the accurate resolution of near-boundary curvature, leading to elevated residuals in those regions.

2. **Near-boundary sampling**: include additional training points in the near-boundary region with increased weight, improving the resolution without altering the variational formulation.

3. **Dynamic penalty**: start with a small $\lambda_{bc,d}$ and increase it gradually during training, preventing the optimization from being dominated by the boundary constraints at early stages.

Benchmark 2: mixed BC (Dirichlet–Neumann), sinusoidal forcing ($f(x) = \sin(\pi x)$), shallow net

– **PDE and BCs.**

We solve

$$-u''(x) = \sin(\pi x), \qquad x \in (0, 1), \qquad u(0) = 0, \qquad u'(1) = g,$$

with $g = 0$ in this run. A closed-form solution is

$$u^*(x) = \frac{\sin(\pi x)}{\pi^2} + \left(g + \frac{1}{\pi}\right) x \quad \Rightarrow \quad u^*(x) = \frac{\sin(\pi x)}{\pi^2} + \frac{1}{\pi} x \;\; (g = 0).$$

– **Model**
Shallow photonic model (2 blocks), mesh size $N = 3$.

– **Training hyperparameters**

epochs	batch	step size	λ_0	N	g
7500	64	0.005	50.0	3	0.0

– **Results**

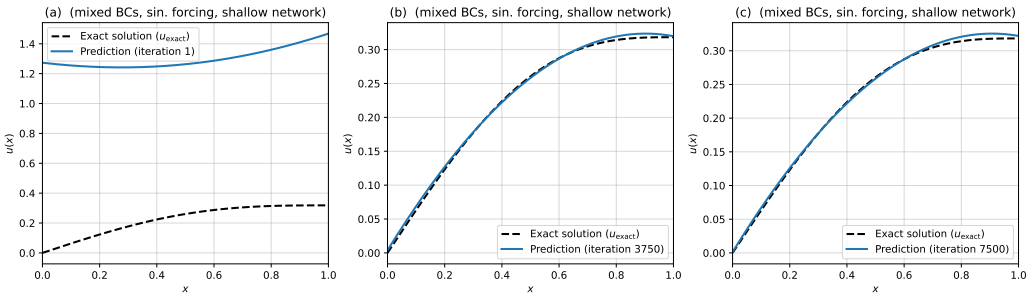

Figure 10: Predicted solution versus the analytical reference.

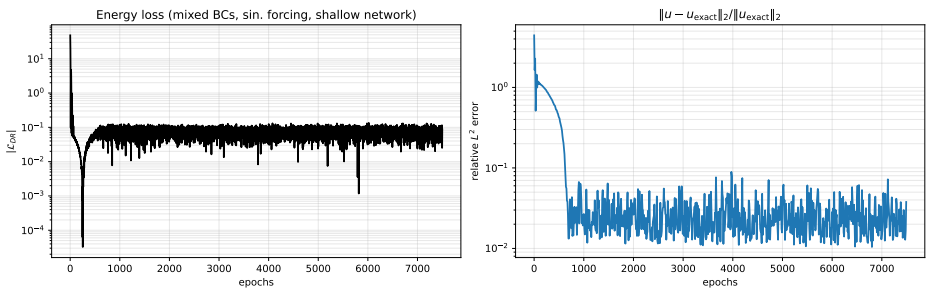

Figure 11: Deep Ritz training loss versus epoch.

Benchmark 3: mixed BC (D–N), sinusoidal forcing ($f(x) = \sin(\pi x)$), deep net

– **PDE and BCs**
Same PDE and boundary conditions as Benchmark 2, including Neumann datum $g = 0$.

– **Model**
Deep photonic model with the same mesh size $N = 3$, but increased depth (number of repeated `ClementsLayer` $\to z^2$ blocks). Here the network depth is set to 4.

– **Training hyperparameters**
The training hyperparameters are *identical* to those of the previous benchmark; only the network depth is changed.

– **Results**

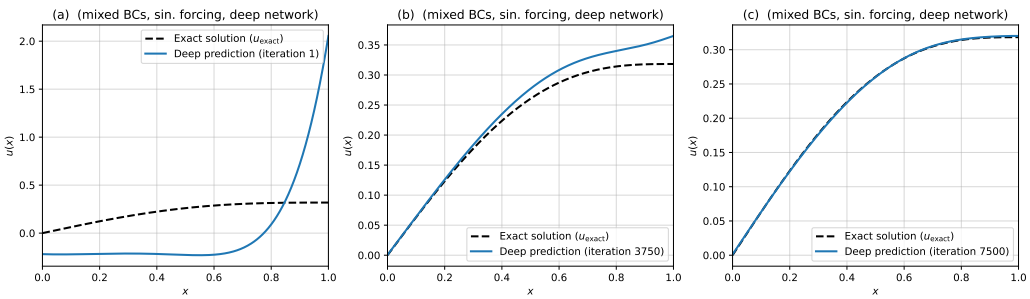

Figure 12: Predicted solution versus the analytical reference.

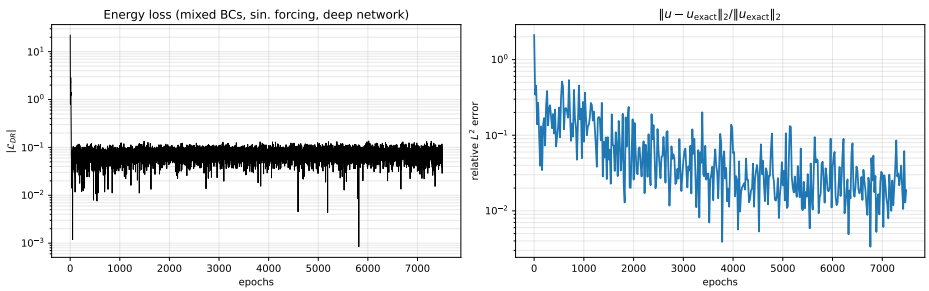

Figure 13: Deep Ritz training loss versus epoch.

– **Effects of depth**
All error values are reported in units of 10^{-2}.

Model	$\lvert\mathcal{L}_{DR}\rvert_{\text{final}}$	$(\text{rel. } \mathcal{L}^2)_{\text{final}}$	$\lvert u_\theta(0)\rvert$	$\lvert u'_\theta(1) - g\rvert$	RMS(r) (interior)
shallow	8.710	3.775	0.125	7.396	29.05
deep	9.504	1.873	0.115	0.507	5.368

Remarks

Both benchmarks achieve good agreement with the analytical reference in relative L^2, but the shallow model still shows a noticeably larger Neumann boundary mismatch and a larger strong-form residual. This is consistent with the B1 discussion: the Deep Ritz objective is a variational (energy) surrogate and its value alone does not guarantee a uniformly small pointwise residual. The B2–B3 comparison isolates the effect of network depth at fixed PDE, boundary conditions, forcing, mesh size N and training hyperparameters. Accordingly, even though the final Deep Ritz energies in Fig. 11 and Fig. 13 are of comparable magnitude and can take negative values, the physics diagnostics reveal a clear separation in PDE fidelity.

In particular, the deeper model achieves a significantly smaller Neumann mismatch and a markedly reduced strong-form residual, together with an improved relative $\mathcal{L}^2$ error. Overall, this supports interpreting the energy loss primarily as a training signal, while residual- and boundary-based diagnostics remain essential to quantify accuracy.

7 Conclusions and future work

7.1 Future work

The Deep Ritz Method (DRM) cannot be applied to arbitrary PDEs. An energy formulation requires, in our setting, a symmetric bilinear form, which naturally restricts the class of problems we can address. For this reason, in this work we focus on elliptic problems (e.g. Poisson) for which the variational formulation is standard and well posed. Problems involving convection typically lead to non-symmetric operators; extending the photonic approach to such cases is left for future work. In this direction, methods such as the Deep Double Ritz framework (saddle-point formulations) provide a principled route to go beyond symmetric settings.

A second limitation is the optimization landscape. Even if the PDE is linear in the unknown u, the parametrization u_θ induces a non-convex optimization problem in the network parameters. This is not specific to DRM: it is a shared feature of most neural solvers. Our contribution is methodological: we demonstrate a stable in-situ training loop on photonic hardware originally developed for supervised tasks, rather than claiming guarantees of global optimality. In practice, common stabilization strategies (learning-rate scheduling, clipping, early stopping) are available, while the potential benefits of light-based surrogate models in mitigating optimization issues require further investigation.

Regarding boundary conditions, Neumann conditions are naturally incorporated into the energy functional, whereas Dirichlet conditions must still be enforced explicitly in the loss (e.g. by lifting techniques or by penalization). In this project we adopt a penalty term (with a ramp schedule in the 2D case) for stability in the in-situ setting; lifting is a well-known alternative that can be integrated in future refinements.

We also emphasize that DRM is not automatically the best choice for low-dimensional problems: classical Galerkin/FEM methods typically outperform neural approaches in accuracy and efficiency. Therefore, our 2D benchmark should be read as a controlled proof-of-concept meant to validate the process pipeline, not as a claim of superiority over FEM. So, it is true that classical Galerkin methods typically outperform the Deep Ritz Method in terms of accuracy and efficiency. However, our goal is to propose a new research trajectory toward easily fabricable, scalable, and more sustainable AI-based solvers for engineering-relevant problems. Large-scale numerical simulations

can require a single run costing thousands of euros, producing significant CO_2 emissions, and demanding substantial computational time. By leveraging photonic computing, we aim to address these challenges by gradually transitioning toward hybrid models that combine classical numerical methods with physical hardware acceleration, while exploiting mature silicon technologies and well-established nanofabrication processes already adopted by industry. So, the long-term motivation is scalability and mesh-free modeling when the dimension increases, where pointwise strong-form enforcement can become unstable and sampling requirements can be severe. A key conceptual advantage of the variational approach is that the PDE is enforced in an average (integral) sense over the domain; empirically, accurate quadrature of such objectives can require fewer samples than pointwise enforcement at comparable accuracy, although this remains problem- and implementation-dependent.

Finally, we clarify a practical aspect of our implementation. We adopt a variational formulation, whose continuous well-posedness requirements involve only first-order derivatives. The lower regularity requirement property therefore refers to the underlying continuous problem and not to the absence of second derivatives in the implementation. In our TRIM-based photonic implementation, the Laplacian $-\Delta u_\theta$ appears only in the computation of the variational derivative used to construct the adjoint source. This does not impose additional regularity on the true solution, since the derivatives are taken with respect to the spatial coordinates of the neural-network surrogate u_θ, which is smooth by construction. Moreover, these derivatives are obtained via structured propagation with respect to the inputs, rather than through generic higher-order automatic differentiation (AD). As a result, potential computational savings arise from avoiding pointwise strong-form enforcement and full higher-order AD, not from the absence of second derivatives at the implementation level. So this second order derivative computation should be viewed as an implementation choice rather than a change in the underlying weak formulation. We potentially reduce computational overhead by avoiding a digital black-box AD pipeline and instead using an architecture-aware recursive derivative propagation in the photonic circuit, with TRIM providing gradients with respect physical parameters. A quantitative comparison against standard AD remains future work.

Another observation that we make is that we neglected the residual connections in our architecture. They help to avoid the *vanishing gradient problem*, making the model easier to train. We leave their utilization and photonic implementation to future work in order to explore other training options.

Future work should also try to implement the other use cases of the Deep-Ritz algorithm showed in [7]. In particular eigenvalue problems could be investigated, as their applications can be many.

As another direction for future work, it would be natural to move beyond real-valued projections and gauge-fixing strategies by formulating fully complex-valued variational objectives, in which both the real and imaginary components of the optical field are directly embedded in the functional, potentially enabling richer regularization mechanisms and improved optimization dynamics in photonic PDE solvers. The idea, indeed, is that knowing that the solution of the PDE is real, we could add a penalty term on the imaginary part of a possibly complex output of the linear readout (in this setting one should stay general and remove the real part operator of our linear readout), avoiding neglecting it as we did in this work. The penalty term could appear in a natural way through a complex extension of our variational formulation, substituting real norms with complex ones and imposing a more general solution of the PDE ($w(x) = u(x) + iv(x)$). The idea is that adding a penalty term on $v(x)$ could help the network's training convergence.

7.2 Conclusions

In this work, we presented a fully in-situ photonic implementation of the Deep Ritz method for the unsupervised solution of partial differential equations. By exploiting adjoint injection, the training signal is generated directly from physics-based energy functionals, without relying on labeled data or digital backpropagation. Through a set of one-dimensional Poisson benchmarks with both homogeneous Dirichlet and mixed Dirichlet–Neumann boundary conditions, we demonstrated that the proposed photonic pipeline correctly captures the weak formulation of the problem and yields stable training dynamics. The results confirm that the variational energy acts as an effective optimization signal, while strong-form residuals and boundary diagnostics remain essential to assess pointwise accuracy. Overall, this work provides a concrete proof of concept for photonic, physics-informed training loops and represents a step toward faster and more sustainable large-scale simulations.

References

[1] I. A. D. Williamson et al. "Reprogrammable Electro-Optic Nonlinear Activation Functions for Optical Neural Networks". In: *IEEE Journal of Selected Topics in Quantum Electronics* 26.1 (2020), pp. 1–12.

[2] Yingheng Tang et al. "Optical neural engine for solving scientific partial differential equations". In: *Nature Communications* (2025). DOI: 10.1038/s41467-025-59847-3. URL: https://doi.org/10.1038/s41467-025-59847-3.

[3] Kirill P. Kalinin et al. "Analog optical computer for AI inference and combinatorial optimization". In: *Nature* (2025). Open access. DOI: 10.1038/s41586-025-09430-z. URL: https://doi.org/10.1038/s41586-025-09430-z.

[4] Hongyi Yuan et al. "Microcomb-driven Photonic Chip for Solving Partial Differential Equations". In: *Advanced Photonics* 7.1 (2025), p. 016007. DOI: 10.1117/1.AP.7.1.016007. URL: https://doi.org/10.1117/1.AP.7.1.016007.

[5] Alfio Quarteroni, Paola Gervasio, and Francesco Regazzoni. "Combining physics-based and data-driven models: advancing the frontiers of research with Scientific Machine Learning". In: *arXiv* (2025). arXiv: 2501.18708 [math.NA].

[6] M. W. M. G. Dissanayake and N. Phan-Thien. "Neural-network-based approximations for solving Partial Differential Equations". In: *Communications in Numerical Methods in Engineering* 10.3 (1994), pp. 195–201.

[7] W. E and B. Yu. "The Deep Ritz Method: A Deep Learning-Based Numerical Algorithm for Solving Variational Problems". In: *Communications in Mathematics and Statistics* 6.1 (2018), pp. 1–12.

[8] Carlos Uriarte. *Solving Partial Differential Equations Using Artificial Neural Networks*. 2024. eprint: arXiv:2403.09001. URL: https://arxiv.org/abs/2403.09001.

[9] Alfio Quarteroni and Alberto Valli. *Numerical Approximation of Partial Differential Equations*. Heidelberg: Springer, 1994.

[10] Kaiming He et al. "Deep Residual Learning for Image Recognition". In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE. 2016, pp. 770–778.

[11] Tyler W. Hughes et al. "Training of photonic neural networks through in situ backpropagation and gradient measurement". In: *Optica* 5.7 (2018), pp. 864–871.

[12] Yichen Shen et al. "Deep learning with coherent nanophotonic circuits". In: *Nature Photonics* 11.7 (2017), pp. 441–446. arXiv: 1610.02365.

[13] David C. Plaut, Steven J. Nowlan, and Geoffrey E. Hinton. *Experiments on Learning by Back Propagation*. Tech. rep. Computer Science Department. Pittsburgh, PA: Carnegie-Mellon University, 1986.

[14] W. Shin and S. Fan. "Choice of the perfectly matched layer boundary condition for frequency-domain Maxwell's equations". In: *Journal of Computational Physics* 231.8 (2012), pp. 3406–3431. DOI: 10.1016/j.jcp.2011.12.011.